



22.401 · Fonaments de Programació
Grau en Ciència de Dades Aplicada
Estudis d'Informàtica, Multimèdia i
Telecomunicació

Fonaments de Programació

Unitat 8: Visualització de dades en Python

Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera, tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que n'heu entès la funcionalitat i explorar-ne els detalls d'implementació. En darrer lloc, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

Per tal de guardar possibles modificacions que feu sobre aquest notebook, us aconsellem que munteu la unitat de Drive a Google Colaboratory (colab). Per fer-ho, heu d'executar les següents instruccions:

```
In [ ]: 1 from google.colab import drive
        2 drive.mount('/content/drive')
```

```
In [ ]: 1 %cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_8
```

Introducció

Anomenem **visualització de dades** la representació i la presentació de dades que fa servir la nostra percepció visual amb l'objectiu de millorar-ne la cognició.

En funció de la intenció amb què es crea la visualització de dades, distingim principalment entre visualitzacions exploratòries i visualitzacions explicatives. Una visualització de dades **exploratòria** permet a l'usuari explorar de manera visual les dades amb què treballa i familiaritzar-s'hi. En aquest cas, normalment el receptor és el creador de la visualització. En canvi, una visualització **explicativa** mira de transmetre informació al receptor de manera intencionada, habitualment mitjançant una narrativa específica. En aquest cas, el receptor de la visualització sol diferir del creador.

Hi ha moltes maneres de visualitzar dades. Quan les dades són quantitatives, poden representar-se en forma de gràfics (per exemple, diagrames de barres, gràfics de línies, gràfics circulars, histogrames, diagrames de dispersió, diagrames de caixa, etc.). A més, si les dades contenen localitzacions es poden crear visualitzacions que incloguin mapes (per exemple, cartogrames), i si les dades representen xarxes es poden visualitzar com a dibuixos de grafs. Quan les dades són qualitatives se solen emprar altres tècniques de visualització, com per exemple, núvols de paraules.

Per tenir una idea més clara dels tipus de visualitzacions de dades que hi ha, en podeu veure alguns exemples recollits a la [guia d'Introducció a la visualització de dades de la Universitat de Duke (http://guides.library.duke.edu/datavis/vis_types)] i en el [Catàleg de visualitzacions](http://www.datavizcatalogue.com/) (<http://www.datavizcatalogue.com/>).

Fer bones visualitzacions no és tasca fàcil: cal tenir en compte diversos factors, com el funcionament del procés de percepció visual o el suport que es farà servir. En general, hi ha un conjunt de bones pràctiques acceptades a l'hora de crear visualitzacions. Per exemple, es considera que la visualització ha de ser tan simple com sigui possible, evitant que introdueixi distraccions que en dificultin la comprensió.

En mòduls anteriors, ja hem vist com podem generar i importar dades, així com alguns mètodes de visualització bàsics de dades. En aquest mòdul ens centrarem en la generació de visualitzacions de dades més avançades. Primer veurem un petit resum de les principals llibreries de visualització de dades que ens trobem a Python tot començant amb la llibreria principal de Python, [Matplotlib](https://matplotlib.org/) (<https://matplotlib.org/>).

Amb més detall, veurem com podem fer visualitzacions amb llibreries com [seaborn](https://seaborn.pydata.org/) (<https://seaborn.pydata.org/>), que ens proveeix d'una interfície d'alt nivell a Matplotlib, amb què podem crear gràfiques atractives amb poques línies de codi, així com altres llibreries que fan servir JavaScript per fer visualitzacions interactives, com [bokeh](https://bokeh.org) (<https://bokeh.org>) i [ipywidgets](https://ipywidgets.readthedocs.io/en/stable/) (<https://ipywidgets.readthedocs.io/en/stable/>), útils per fer *dashboards* o quadres de comandament de dades.

Veurem tot seguit com podem representar dades de xarxes en forma de grafs amb la llibreria [Networkx](https://networkx.github.io/) (<https://networkx.github.io/>).

Finalment, veurem com podem representar dades espacials sobre mapes amb la llibreria [folium](https://python-visualization.github.io/folium/) (<https://python-visualization.github.io/folium/>) i amb l'ús de [GeoPandas](https://geopandas.org/en/stable/) (<https://geopandas.org/en/stable/>).

A més de les llibreries que acabem d'esmentar, durant aquest mòdul farem servir també les llibreries que ja hem anat presentant als mòduls anteriors: [NumPy](http://www.numpy.org/) (<http://www.numpy.org/>), [pandas](http://pandas.pydata.org/) (<http://pandas.pydata.org/>) i [scikit-learn](http://scikit-learn.org) (<http://scikit-learn.org>).

[1 Un mapa de les llibreries de visualització de Python](#)

[1.1 Visualització d'alt nivell](#)

[1.2 Visualització científica](#)

[1.3 Visualització de grafs](#)

[1.4 Visualització de geolocalització](#)

[1.5 Visualització interactives](#)

[2 Visualització d'alt nivell](#)

[2.1 Matplotlib](#)

[2.1.1 Introducció](#)

[2.1.2 Estructura de les figures amb Matplotlib](#)

[2.1.3 Recull d'exemples](#)

[2.2 Altair](#)

[2.3 plotnine](#)

[2.4 Gràfiques amb Seaborn](#)

[3 Visualització de Grafs amb Networkx](#)

[4 Visualització amb geolocalització](#)

[4.1 GeoJSON](#)

[4.2 geopandas](#)

[4.2.1 Àrees](#)

[4.2.2 Centroides i càlcul de distàncies/a>](#)

[4.3 Folium](#)

[5 Visualització interactiva i creació de quadres de comandament o dashboards](#)

[5.1 bokeh](#)

[5.2 ipywidgets](#)

[5.2.1 interact](#)

[5.2.2 Múltiples arguments](#)

[5.2.3 Crear instàncies de widgets](#)

[5.2.4 Creant interfícies d'usuari](#)

[5.2.5 Botons](#)

[6 Exercicis i preguntes teòriques](#)

[6.1 Instruccions importants](#)

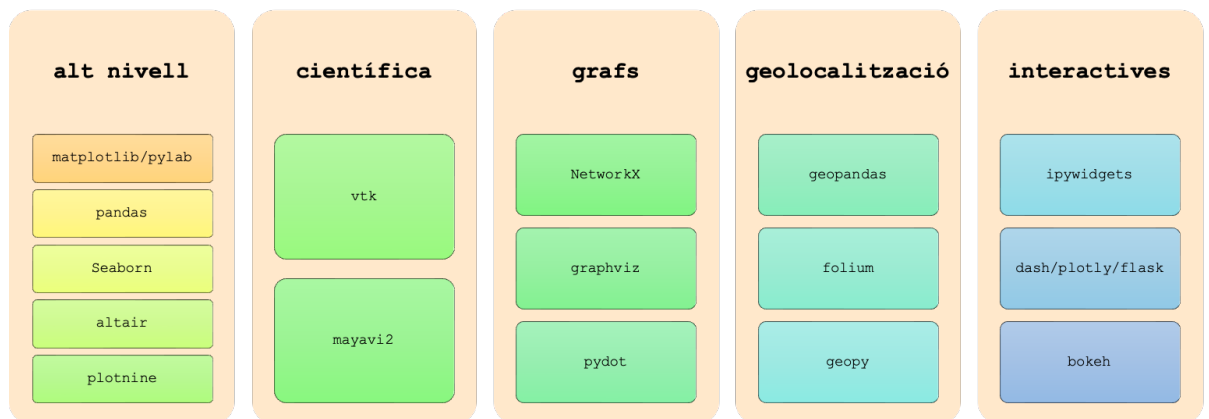
[7 Bibliografia](#)

1 Un mapa de les llibreries de visualització de Python

En aquest apartat donarem una petita visió de les principals llibreries per visualitzar dades. No totes les explicarem amb detall, però sí que en donarem potser algun exemple i algunes referències.

Les llibreries de visualització es poden agrupar en funció del seu objectiu o especialització (per exemple, si es fan servir per representar dades, superfícies o grafs), o bé pels seus nivells d'abstracció (de baix nivell, si contenen funcions per treballar a baix nivell, i controlar corbes i detalls gràfics, o bé si són capaces de representar objectes complexos.)

Un mapa de les llibreries que comentarem en aquest mòdul didàctic, a part de Matplotlib, són:



En aquesta secció farem una llista ràpida describint què és cada llibreria i per què pot ser útil, seguint la categorització del diagrama.

1.1 Visualització d'alt nivell

Respecte dels projectes de representació de figures d'alt nivell, podem comentar els quatre següents de manera singular:

1. [Matplotlib](http://matplotlib.org/) (<http://matplotlib.org/>) és una llibreria de representació de dades en 2D molt emprada per la comunitat atesa la gran qualitat de les figures generades i la seva capacitat de representació de dades de diferent índole. Dintre de Matplotlib, *pylab*, és la interfície d'alt nivell que permet crear gràfics amb un codi i funcionament molt similar a com es faria en Matlab.
2. Potser tot seguit cal comentar [Seaborn](https://seaborn.pydata.org/) (<https://seaborn.pydata.org/>), dissenyada per crear gràfiques a partir d'objectes `pandas` (no només), i que descriurem a l'apartat següent.
3. Una tercera llibreria d'alt nivell interessant és [altair](https://github.com/altair-viz/altair) (<https://github.com/altair-viz/altair>). Altair és una llibreria per programar gràfics de manera [declarativa](https://en.wikipedia.org/wiki/Declarative_programming) (https://en.wikipedia.org/wiki/Declarative_programming). Es basa en la seva visualització en una llibreria de javascript anomenada [Vega-Lite](https://github.com/vega/vega-lite) (<https://github.com/vega/vega-lite>).
4. Plotnine (i ggplot). [plotnine](https://plotnine.readthedocs.io/en/stable/) (<https://plotnine.readthedocs.io/en/stable/>) és una implementació d'una gramàtica de gràfics en Python, es basa en [ggplot2](https://ggplot2.tidyverse.org) (<https://ggplot2.tidyverse.org>), una llibreria molt emprada i popular en el llenguatge de programació estadística [R](https://www.r-project.org) (<https://www.r-project.org>).

1.2 Visualització científica

Altres llibreries estan especialitzades en visualització d'un altre tipus, potser menys útils en el camp de la ciència de dades. Una llibreria interessant i que es fa servir molt per visualització científica es VTK.

El [kit d'eines de visualització \(VTK\)](https://vtk.org/about/#overview) (<https://vtk.org/about/#overview>) és un sistema de programari de codi obert i disponible de franc per a gràfics per a ordinador en 3D, modelatge, processament d'imatges, renderització de volums, visualització científica i traçat en 2D.

La llibreria admet una gran varietat d'algoritmes de visualització i tècniques de modelatge avançades, i aprofita el processament en paral·lel i distribuït per a la velocitat i l'escalabilitat, respectivament. Una llibreria semblant i relacionada és [Mayavi2](https://docs.entthought.com/mayavi/mayavi/) (<https://docs.entthought.com/mayavi/mayavi/>). Aquesta llibreria és una eina multiplataforma de propòsit general per a la visualització de dades científiques en 3D.

1.3 Visualització de grafs

En teoria de grafs, un graf és una representació abstracta d'un conjunt d'objectes en què alguns parells dels objectes estan connectats per enllaços o arestes.

Els objectes interconnectats els anomenem vèrtexs i poden representar qualsevol cosa de manera abstracta. Els enllaços que connecten alguns parells de vèrtexs s'anomenen arestes.

Típicament, un graf es descriu de manera esquemàtica com a conjunt de punts o cercles per als vèrtexs, units per línies o corbes per les arestes. Els grafs són un dels objectes d'estudi de la matemàtica discreta.

En Python tenim potser tres llibreries importants per representar grafs:

- `networkx` . [NetworkX \(https://networkx.org\)](https://networkx.org) és un paquet Python per a la creació, manipulació i estudi de l'estructura, la dinàmica i les funcions de xarxes complexes. Aquest paquet l'estudiarem amb més detall dintre d'aquest mòdul didàctic.
- `graphviz` . [Graphviz \(https://graphviz.org\)](https://graphviz.org) és un programari de visualització de gràfics de codi obert a què també es pot accedir mitjançant Python. La visualització de gràfics és una manera de representar la informació estructural com a diagrames de gràfics i xarxes abstractes. Té aplicacions importants en xarxes, bioinformàtica, enginyeria de programari, disseny de bases de dades i web, aprenentatge automàtic i en interfícies visuals per a altres dominis tècnics.
- `pydot` . [Pydot \(https://pypi.org/project/pydot/\)](https://pypi.org/project/pydot/) és la llibreria en Python que ens permet interaccionar amb Graphviz

1.4 Visualització de geolocalització

Hi ha força llibreries de visualització i ús de geolocalització en Python, potser les que es fan servir més són:

- `geopandas` . Com el seu nom indica, [geopandas \(https://geopandas.org\)](https://geopandas.org) és una extensió de `pandas` per incloure informació de geolocalització.
- `folium` . Potser més centrada en la part de visualització, [folium \(https://python-visualization.github.io/folium/\)](https://python-visualization.github.io/folium/) es basa en la biblioteca de javascript leaflet.js. Permet la manipulació i visualització interactiva.
- `geopy` . [geopy \(http://geopy.readthedocs.io\)](http://geopy.readthedocs.io) és un client Python que permet interaccionar amb diversos serveis web de geocodificació populars. `geopy` facilita localitzar les coordenades d'adreces, ciutats, països i punts de referència arreu del món mitjançant geocodificadors de tercers i altres fonts de dades. També inclou eines per calcular distàncies, canviar unitats i d'altres.

1.5 Visualitzacions interactives

Un aspecte extraordinàriament útil de l'ús de Python per a ciència de dades és la relativa facilitat de crear visualitzacions interactives amb certs elements d'interfície d'usuari d'una manera ràpida.

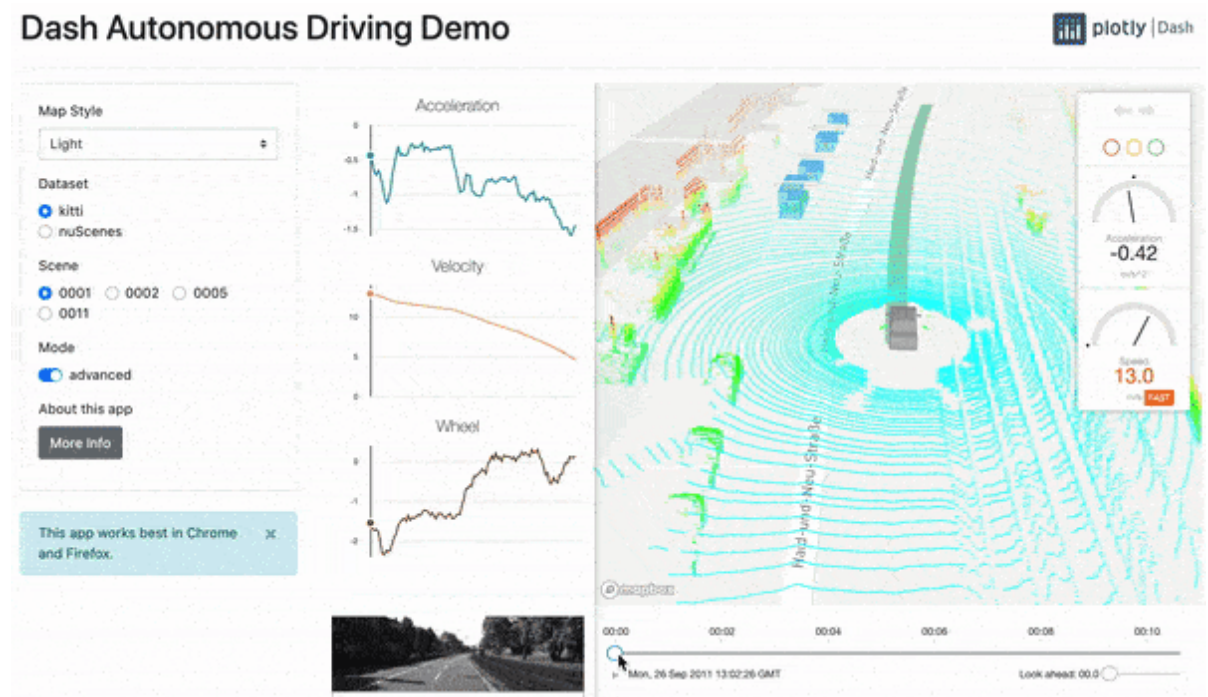
Aquestes visualitzacions es poden generar per servir en un programa de Python, es poden generar i integrar en un servei web (i accedir-hi mitjançant un navegador) o bé integrar-les en un notebook de jupyter .

D'aquestes últimes, potser podem esmentar de manera notable `dash` , `bokeh` i `ipywidgets` . Tant `bokeh` com `ipywidgets` els veurem amb detall en aquesta unitat, però és interessant saber alguna cosa més de `dash` .

`dash` (<https://plotly.com/dash/>) és una llibreria relacionada amb `plotly` (<https://plotly.com>) que permet la creació de dashboards o taulers de control amb orientació a la ciència de dades i interactivitat.

La part potser més interessant de `dash` és que és una extensió d'un marc de programació web anomenat `flask` (<https://flask.palletsprojects.com/en/2.0.x>), de manera que permet la inclusió d'estris de visualització interactiva proporcionats per `plotly` d'una manera ben fàcil.

`Flask` és una via interessant d'elaborar pàgines web i serveis API amb Python. Juntament amb `dash`, es poden crear dashboards que permetin fer visualitzacions de fons i bases de dades interactivament en una web mitjançant només Python. Potser els detalls d'aquesta llibreria cauen fora del conjunt de materials d'aquesta assignatura, però és interessant que sapiguem que existeix.



Exemple d'un *dashboard* creat amb `dash` i `flask` per mostrar les dades d'un sistema de conducció autònoma

2 Visualitzacions d'alt nivell

Veurem ara amb més detall algunes de les llibreries que hem esmentat. Cal dir que no les veurem totes, així doncs, hem organitzat la resta del material començant per les llibreries que no estan especialitzades amb visualitzacions interactives d'alt nivell.

Començarem descrivint breument exemples amb Matplotlib, altair i plotnine, i veurem amb més detall seaborn.

2.1 Matplotlib

2.1.1 Introducció

[Matplotlib \(http://matplotlib.org/\)](http://matplotlib.org/) és una llibreria de representació de dades en 2D molt emprada per la comunitat atesa la gran qualitat de les figures creades i la seva capacitat de representació de dades d'índole diferent. Un cop d'ull ràpid a la [galeria d'exemples \(http://matplotlib.org/gallery.html\)](http://matplotlib.org/gallery.html) ens convencerà de la gran potència que té.

Aquesta llibreria es pot fer servir o bé mitjançant la interfície Pyplot, que s'integra molt bé amb IPython i Notebook i té una sintaxi molt similar a [MATLAB \(http://es.mathworks.com/products/matlab/\)](http://es.mathworks.com/products/matlab/), o bé amb els objectes que proporciona la llibreria per fer servir tota la seva capacitat de personalització de dibuix.

Aquesta similitud amb MATLAB ha ajudat a la seva disseminació, ja que molts usuaris científics buscaven una alternativa de codi obert enfront de MATLAB i que pogués estar integrada amb altres llibreries escrites en Python.

El codi de Matplotlib està dividit en tres parts: *pylab*, *Matplotlib API* i *backends*. La primera part, *pylab*, és la interfície que permet crear gràfics amb un codi i funcionament molt similar a com es faria en Matlab. *Matplotlib API* és la part essencial que la resta de codi fa servir i, finalment, *backends* és la part encarregada de la representació depenent de la plataforma (tipus de fitxers d'imatge, dispositius de visualització, etc.). En aquest mòdul només farem exemples i exercicis fent servir *pylab*.

Podeu consultar molts exemples a l'[ajuda de la llibreria \(https://matplotlib.org/stable/gallery/index\)](https://matplotlib.org/stable/gallery/index).

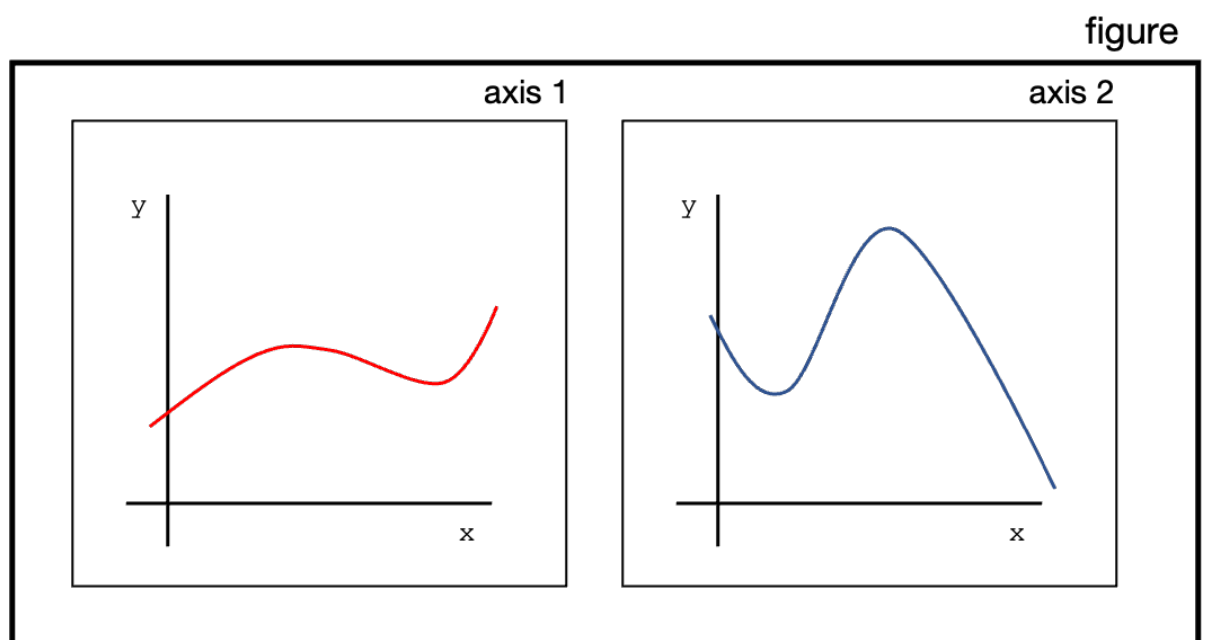
2.1.2 Estructura de les figures amb Matplotlib

Una gràfica a Matplotlib conté sobretot dos objectes principals:

- Figures. Les figures (`figure`) són l'espai en què podrem ubicar una o més gràfiques o objectes.
- Eixos. Eixos (`axis`), en referència a cadascun dels objectes que posarem a dintre d'una figura.

D'aquesta manera, hem de pensar en una figura com un espai en què es delimita la visualització que volem fer, i els eixos com cadascun dels objecte que hi volem ubicar.

Una figura conté un mínim d'un eix.



Podem invocar la nostra primera figura mitjançant `pyplot`. Veureu que també incloem una directiva pel Jupyter notebook perquè entengui que estem creant les gràfiques a un notebook.

```
In [7]: 1
        2 %matplotlib notebook
        3
        4 import matplotlib.pyplot as plt
```

Ara veurem com l'objecte `plt` el podem fer servir per elaborar una figura, en aquest cas sense que hi hagi res. Per exemple:

```
In [8]: 1 fig = plt.figure()  
        2 fig.show();
```

<IPython.core.display.Javascript object>

Com comentàvem, podem pensar en la figura com un llenç en què podem dibuixar, aquest llenç pot tenir les dimensions que creguem convenient amb l'argument `figsize`.

A partir d'aquí, podem fer servir la comanda `plot()` per crear la nostra primera corba i veure com podem afegir text als eixos `x` i `y`, i un títol.

```
In [9]: 1 fig = plt.figure(figsize=(3,3))
        2 plt.plot((1,2,4), (1,2,6))
        3 plt.xlabel("x")
        4 plt.ylabel("y")
        5 plt.title("My figure")
        6 fig.show();
```

<IPython.core.display.Javascript object>

Podem incloure més eixos en una figura, tot fent servir la comanda `subplots`. Fixeu-vos, en aquest cas, com creem una figura amb 4 eixos, en una disposició de 2x2, en què ja tenim descriptors diferents per a la mateixa figura (`fig`), i per a cadascun dels eixos (`axs`):

```
In [10]: 1 fig, (axs) = plt.subplots(2,2)
          2 fig.show()
```

<IPython.core.display.Javascript object>

Podem ara posar la nostra gràfica o `plots` en qualsevol dels eixos (continguts a la variable `axs`). Podem, tot fent servir el descriptor d'un eix en particular, col·locar figures o modificar propietats d'aquest eix en particular.

```
In [11]: 1 fig, (axs) = plt.subplots(2,2)
2         axs[0,1].plot((0,2,4), (0,2,6),"b")
3         axs[0,1].set(title="Figure in 0,1",
4                       xlabel="x",
5                       ylabel="y")
6         axs[1,0].plot((1,3,4), (2,-1,6),"r")
7         axs[1,0].set(title="Figure in 1,0",
8                       xlabel="x",
9                       ylabel="y")
10        fig.show()
```

<IPython.core.display.Javascript object>

Si tornem al cas de tenir una figura, la podem crear de manera idèntica, i modificar, per exemple, propietats de les etiquetes, i rotar-les 45 graus:

```
In [12]: 1 temperatures = [4, 7, 9, 12,
2                       8, 12, 18, 25,
3                       21, 12, 10, 8]
4
5         months = ["Jan", "Feb", "Mar",
6                  "Apr", "May", "June",
7                  "July", "Aug", "Sept",
8                  "Oct", "Nov", "Dec"]
9
```

```
In [13]: 1 fig, axs = plt.subplots()
2         axs.bar(months,
3                 temperatures,
4                 color="lightblue")
5         plt.setp(axs.get_xticklabels(), rotation = 45)
6         axs.set(title="Monthly temperature",
7                 xlabel = "Month",
8                 ylabel = "T(C)")
9         plt.show()
```

<IPython.core.display.Javascript object>

Finalment, també és normal incloure llegendes en el cas que tinguem diverses corbes en una mateixa figura. En aquest cas, podem controlar l'etiqueta de cada corba amb `label` i la inclusió de la llegenda invocant el mètode `.legend()` al descriptor d'eixos:

```
In [14]: 1 temperaturesA = [4, 7, 9, 12,  
2                 8, 12, 18, 25,  
3                 21, 12, 10, 8]  
4 temperaturesB = [5, 9, 14, 14,  
5                 15, 16, 17, 16,  
6                 15, 8, 2, -5]  
7 months = ["Jan", "Feb", "Mar",  
8           "Apr", "May", "June",  
9           "July", "Aug", "Sept",  
10          "Oct", "Nov", "Dec"]
```



```
In [15]: 1 fig, axs = plt.subplots()
2         axs.plot(months,
3                 temperaturesA,
4                 color="blue",
5                 label="A")
6         axs.plot(months,
7                 temperaturesB,
8                 color="red",
9                 label="B")
10        axs.legend()
11        plt.setp(axs.get_xticklabels(), rotation = 45)
12        axs.set(title="Monthly temperature",
13                xlabel = "Month",
14                ylabel="T(C)")
15        plt.show()
```

<IPython.core.display.Javascript object>

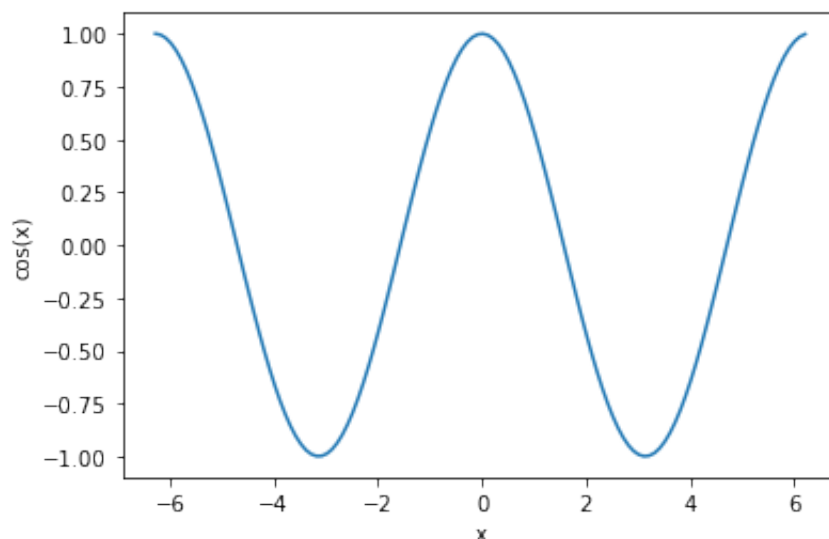
2.1.3 Recull d'exemples

Exemple 1: representar la funció cosinus

Al primer exemple representarem dos `_arrays_`, un davant de l'altre, als eixos X i Y respectivament. Com hem comentat abans, podem aprofitar la manera de generar gràfiques amb una certa interactivitat en el notebok amb `%Matplotlib notebook`, o bé amb `%Matplotlib inline` si les volem no interactives.

In [16]:

```
1 %matplotlib inline
2 # Aquest primer import és necessari per inicialitzar l'entorn d
3 import matplotlib
4 import numpy as np
5 # Importem la llibreria fent servir l'àlies 'plt'.
6 import matplotlib.pyplot as plt
7
8 # Calculem una matriu de  $-2\pi$  a  $2\pi$  amb un pas de 0.1.
9 x = np.arange(-2*np.pi, 2*np.pi, 0.1)
10
11 # Representem l'array x amb el valor de  $\cos(x)$ .
12 plt.plot(x, np.cos(x))
13
14 # Afegim els noms als eixos X i Y respectivament:
15 plt.xlabel('x')
16 plt.ylabel('cos(x)')
17
18 # Finalment, mostrarem el gràfic:
19 plt.show()
```



Exemple 2: representar les funcions cosinus i sinus alhora

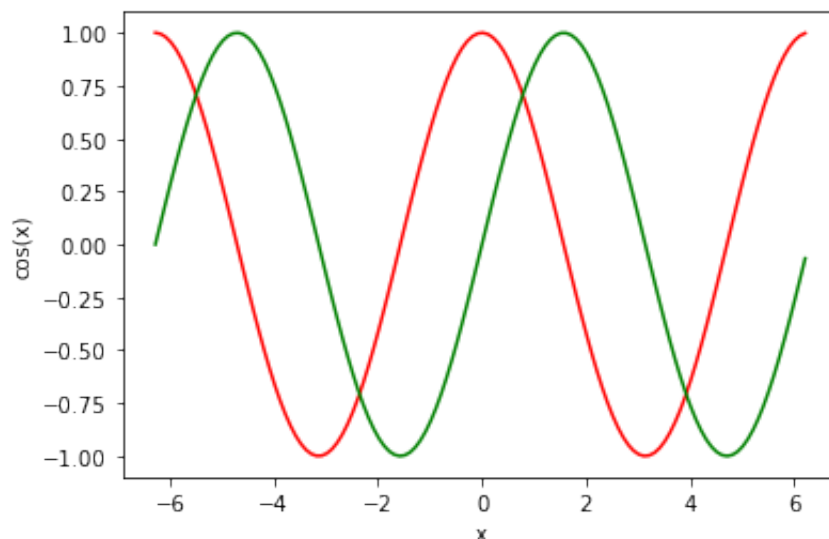
En aquest exemple, calcularem els valors de les funcions sinus i cosinus per al mateix rang de valors i les representarem al mateix gràfic.

In [17]:

```

1  import matplotlib
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  # Calculem una matriu de  $-2\pi$  a  $2\pi$  amb un pas de 0.1.
7  x = np.arange(-2*np.pi, 2*np.pi, 0.1)
8
9  # Podem encadenar la representació de múltiples funcions al mateix gràfic.
10 # En aquest ordre: array x, cos(x), 'r' farà servir el color vermell
11 # array x, sin(x) i verd (green)
12 plt.plot(x, np.cos(x), 'r', x, np.sin(x), 'g')
13
14 # De manera equivalent, podríem fer una crida en dues ocasions:
15 #plt.plot(x, np.cos(x), 'r')
16 #plt.plot(x, np.sin(x), 'g')
17
18 # Afegim els noms als eixos X i Y respectivament:
19 plt.xlabel('x')
20 plt.ylabel('cos(x)')
21
22 # Finalment, mostrarem el gràfic.
23 plt.show()

```

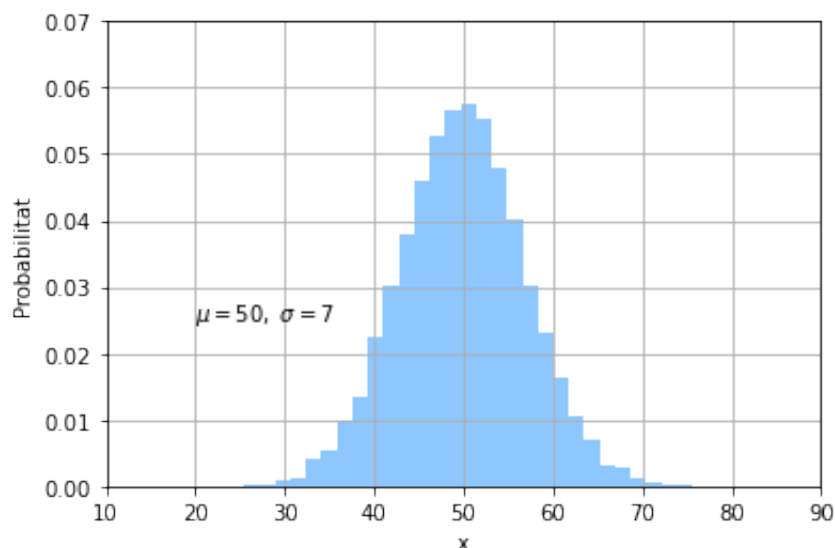
*Exemple 3: histogrames*

Matplotlib disposa de molts tipus de gràfics implementats, entre els quals hi ha els histogrames. En aquest exemple, representem una [funció gaussiana](https://es.wikipedia.org/wiki/Funci%C3%B3n_gaussiana) (https://es.wikipedia.org/wiki/Funci%C3%B3n_gaussiana).

```

In [18]: 1 import matplotlib
          2 import numpy as np
          3 import matplotlib.pyplot as plt
          4 import matplotlib.mlab as mlab
          5
          6 # Paràmetres de la funció gaussiana:
          7 mu, sigma = 50, 7
          8
          9 # Generem una matriu fent servir aquests paràmetres i nombres a
10 x = mu + sigma * np.random.randn(10000)
11
12 # La funció 'hist' calcula la freqüència i el nombre de barres.
13 # normalitza els valors de probabilitat ([0,1]), 'facecolor' co
14 # i 'alpha', el valor de la transparència de les barres.
15 n, bins, patches = plt.hist(x, 30, density=1, facecolor='dodger
16
17 plt.xlabel('x')
18 plt.ylabel('Probabilitat')
19
20 # Situem el text amb els valors de 'mu' i 'sigma' al gràfic:
21 plt.text(20, .025, r'$\mu=50, \sigma=7$')
22
23 # Controlem manualment la mida dels eixos. Els dos primers valo
24 # 'xmax', i els següents, amb 'ymin' i 'ymax':
25 plt.axis([10, 90, 0, 0.07])
26
27 # Mostrem una reixeta:
28 plt.grid(True)
29
30 plt.show()

```



Exemple 4: representació del conjunt de Mandelbrot

El conjunt de Mandelbrot és un dels conjunts fractals més estudiats i coneguts. Podeu trobar més informació en línia sobre [el conjunt i els fractals en general](https://es.wikipedia.org/wiki/Conjunto_de_Mandelbrot) (https://es.wikipedia.org/wiki/Conjunto_de_Mandelbrot).

L'exemple és una adaptació d'[aquest codi](https://scipy-lectures.github.io/intro/numpy/exercises.html#mandelbrot-set) (<https://scipy-lectures.github.io/intro/numpy/exercises.html#mandelbrot-set>).

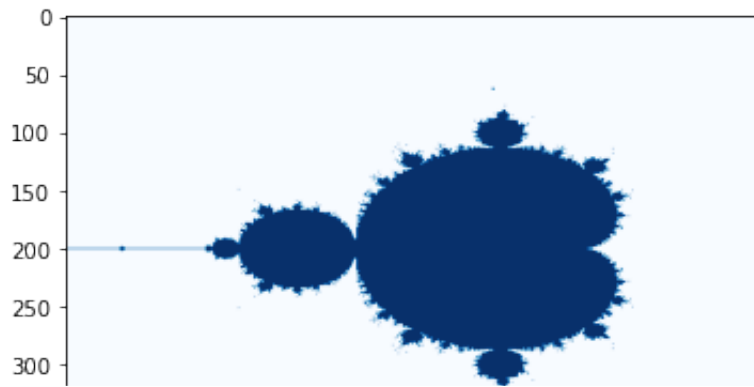
```
In [19]: 1 import numpy as np
2 import matplotlib.pyplot as plt
3 from numpy import newaxis
4
5 # La funció que calcularà el conjunt de Mandelbrot.
6 def mandelbrot(N_max, threshold, nx, ny):
7     # Creem una matriu amb nx elements entre els valors -2 i 1.
8     x = np.linspace(-2, 1, nx)
9     # Fem el mateix, però, en aquest cas, entre -1.5 i 1.5, de
10    y = np.linspace(-1.5, 1.5, ny)
11
12    # Creem el pla de nombres complexos necessari per calcular
13    c = x[:,newaxis] + 1j*y[newaxis,:]
14
15    # Iteració per calcular el valor d'un element en la success.
16    z = c
17    for j in range(N_max):
18        z = z**2 + c
19
20    # Finalment, calculem si un element pertany o no al conjunt
21    conjunto = (abs(z) < threshold)
22
23    return conjunto
24
25 conjunto_mandelbrot = mandelbrot(50, 2., 601, 401)
26
27 # Transposem els eixos del conjunt de Mandelbrot calculat fent
28 # Fem servir la funció 'imshow' per representar una matriu com
29 # 'color map' i és l'escala de colors en què representarem la n
30 # altres mapes a la documentació oficial:
31 # http://matplotlib.org/examples/color/colormaps_reference.html
32 plt.imshow(conjunto_mandelbrot.T, cmap='Blues')
33
34 plt.show()
```

```
/var/folders/j9/mtv_btx92qjcytlqq7fq8r940000gn/T/ipykernel_63924/3
537489133.py:18: RuntimeWarning: overflow encountered in square
```

```
z = z**2 + c
```

```
/var/folders/j9/mtv_btx92qjcytlqq7fq8r940000gn/T/ipykernel_63924/3
537489133.py:18: RuntimeWarning: invalid value encountered in squa
re
```

```
z = z**2 + c
```

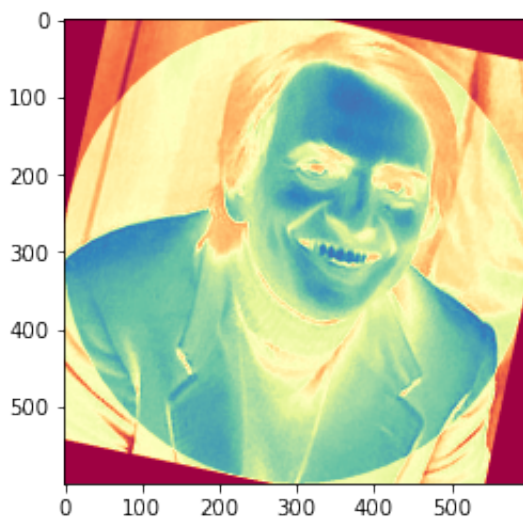
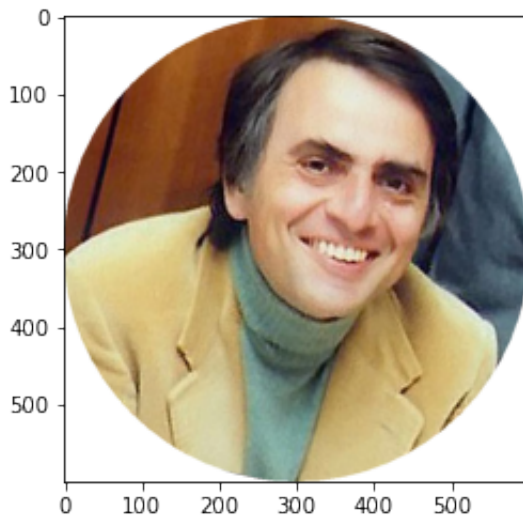


Exemple 5: manipulació d'imatges

Una imatge pot assimilar-se a una matriu multidimensional en què pels valors de píxels (x, y) tenim calors de color. Matplotlib ens permet llegir imatges, manipular-les i aplicar-hi diferents mapes de colors a l'hora de representar-les. A l'exemple següent, carregarem una fotografia en format PNG de Carl Sagan.

Crèdits de la foto: NASA JPL

```
In [20]: 1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.image as mpimg
4
5 # Llegim la imatge mitjançant la funció 'imread'.
6 carl = mpimg.imread('media/sagan.png')
7
8 # Podem mostrar la imatge:
9 plt.imshow(carl)
10 plt.show()
11
12 # I podem modificar els valors numèrics de color llegits per la
13 # Obtenim els valors de l'escala de grisos i mostrem els valors
14 # colors espectrals.
15 gris = np.mean(carl, 2)
16 plt.imshow(gris, cmap='Spectral')
17 plt.show()
```



2.2 Altair

En aquesta llibreria, l'usuari només necessita especificar com s'assignen les columnes de dades al canal de visualització, és a dir, declarar enllaços entre les columnes de dades i els canals de codificació com els eixos, fila, columnes, etc.

En aquest mòdul didàctic, no tractarem Altair amb detall, però potser un exemple de com fer servir `altair`, tot emprant el codi següent:

```
In [21]: 1 import pandas as pd
          2 import altair as alt
          3 import seaborn as sns
          4
          5 # load a simple dataset as a pandas DataFrame,
          6 # from seaborn in this case
          7
          8 df = sns.load_dataset("iris")
          9 df.head()
```

```
Out [21]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [22]: 1 alt.Chart(df).mark_point().encode(
          2     x='sepal_length',
          3     y='sepal_width',
          4     color='species',
          5 )
          6
```

```
Out [22]:
```

Una propietat interessant d' `altair` és que pot generar figures interactives de manera natural. Veureu que podem crear una selecció de dades basades en una selecció directa sobre la figura.

A tall d'exemple, proveu aquest codi i observeu com es modifiquen les propietats de la figura de barres tot seleccionant mostres en la figura de punts (tot dibuixant un recuadre dintre de la figura).

Observeu també com l'avaluació d'un plot és, de fet, invocar-ne la variable:


```
In [23]: 1 source = df
2         brush = alt.selection(type='interval')
3
4         points = alt.Chart(source).mark_point().encode(
5             x='sepal_length',
6             y='sepal_width',
7             color=alt.condition(brush, 'species', alt.value('lightgray'))
8         ).add_selection(
9             brush
10        )
11
12        bars = alt.Chart(source).mark_bar().encode(
13            y='species',
14            color='species',
15            x='count(species)'
16        ).transform_filter(
17            brush
18        )
19
20        points & bars
21
22
23
```

Out [23]:

2.3 plotnine

En el cas de plotnine, la gramàtica permet als usuaris compondre una gràfica de manera incremental i a capes, tot assignant dades de manera semblant al cas anterior.

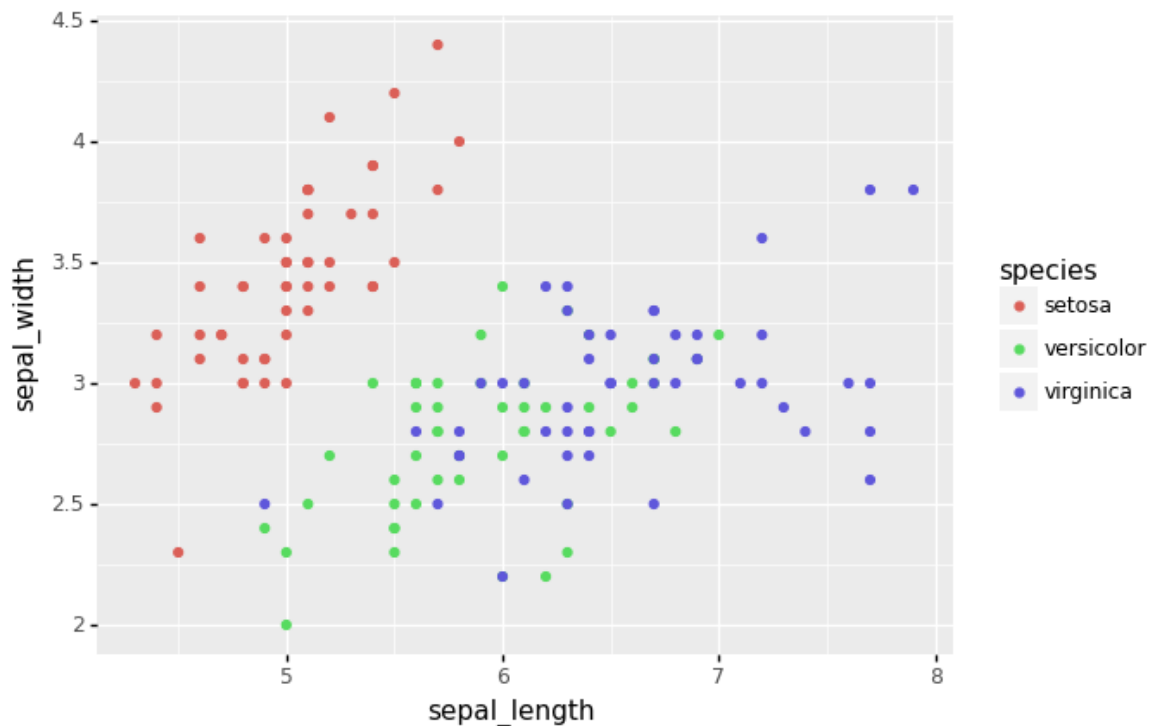
Fer figures amb una gramàtica és un recurs molt útil, fa que les figures personalitzades (i, d'altra banda, complexes) siguin fàcils de pensar i després crear, mentre que les trames simples continuen sent simples.

A tall d'exemple, en aquest cas farem servir el mateix conjunt de dades que en el cas anterior, que ja està inclòs a dintre del mateix paquet, també en format pandas:

```
In [24]: 1 import numpy as np
2         import pandas as pd
3
4         import plotnine
5         from plotnine import *
```

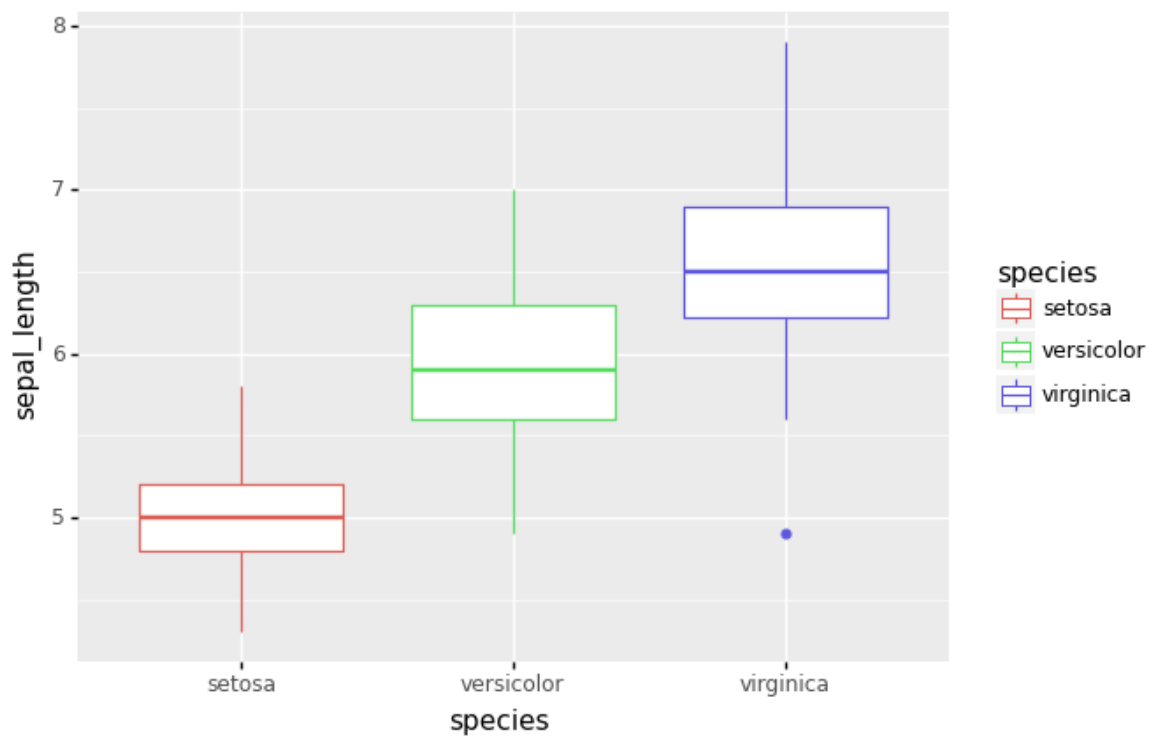
Es pot veure com funciona la gramàtica, en el sentit que amb l'operador + anirem dictant el tipus de plot, o les modificacions al plot. Per exemple, per fer un scoreplot, tot indicant color i coordenades, primer assignem les variables i després indiquem el tipus de plot:

```
In [25]: 1 p = ggplot(df, aes(  
2         x= "sepal_length",  
3         y= "sepal_width",  
4         color = "species")) + geom_point()  
5 p.draw();
```



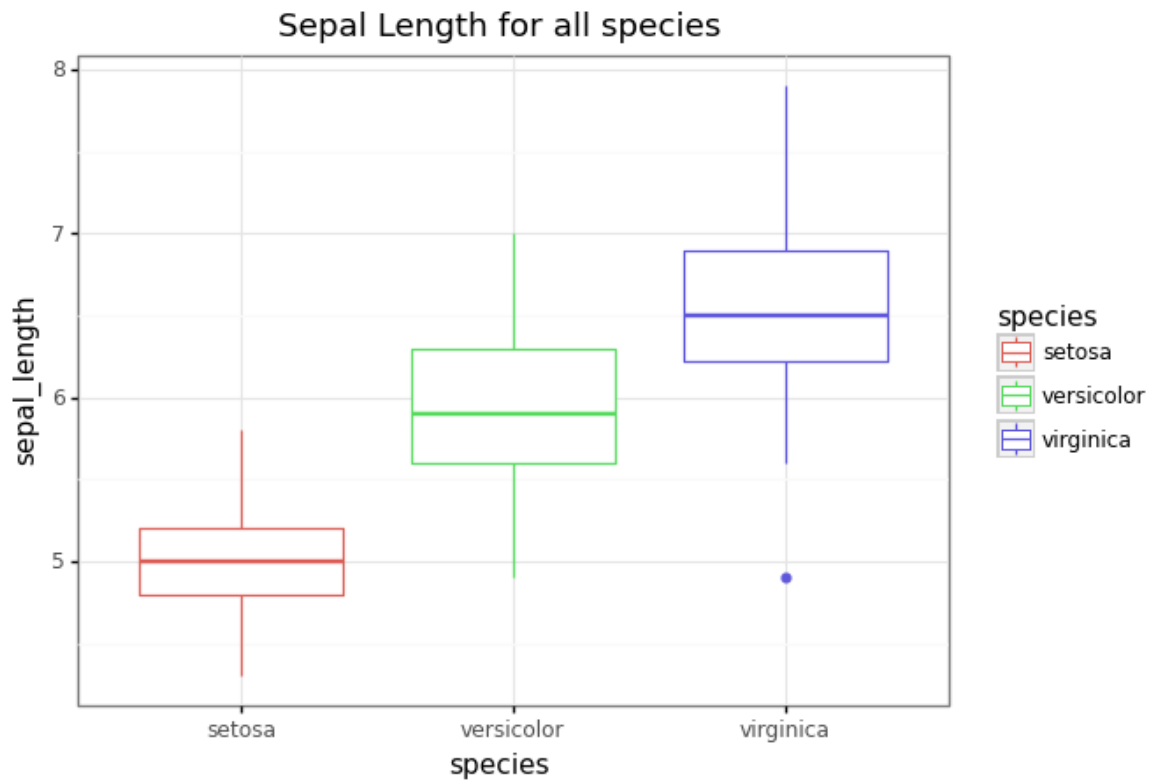
Així, de manera similar a la forma semàntica que ggplot2 fa servir en el llenguatge R, podem modificar fàcilment l'ordre per fer un boxplot en funció del tipus de mostra, o canviar el títol, color o tema, per exemple:

```
In [26]: 1 p = ggplot(df, aes(  
2         y = "sepal_length",  
3         x = "species",  
4         color = "species")) + geom_boxplot()  
5 p
```



```
Out[26]: <ggplot: (373460692)>
```

```
In [27]: 1 p + ggtitle("Sepal Length for all species") + theme_bw()
```



```
Out[27]: <ggplot: (373518315)>
```

2.4 Gràfiques amb Seaborn

Abans ja hem introduït la llibreria de representació de dades en 2D Matplotlib. En aquest apartat, veurem una llibreria de més alt nivell que ens ofereix una interfície a Matplotlib, a més d'altres llibreries que ens permeten crear tipus de visualitzacions addicionals.

[Seaborn \(https://seaborn.pydata.org/\)](https://seaborn.pydata.org/) és una llibreria de visualització basada en Matplotlib que pot treballar amb estructures de dades de Numpy i Pandas. Ens ofereix una interfície d'alt nivell per dibuixar gràfiques estadístiques atractives. Entre altres funcionalitats, Seaborn ens ofereix un conjunt de temes que milloren l'aparença estètica de les gràfiques generades, eines per gestionar paletes de colors, funcions per visualitzar distribucions i funcions per crear matrius de gràfiques que permeten elaborar visualitzacions complexes de manera senzilla.

Atès que Seaborn està basada en Matplotlib, les gràfiques generades amb Seaborn poden acabar d'ajustar-se després amb les eines que ofereix Matplotlib.

Us recomanem que visiteu la [galeria d'exemple de Seaborn \(https://seaborn.pydata.org/examples/index.html#example-gallery\)](https://seaborn.pydata.org/examples/index.html#example-gallery), en què podreu apreciar la gran quantitat de tipus de gràfiques disponibles i els resultats estètics que produeix.

En primer lloc, veurem alguns exemples de generació de gràfiques amb la llibreria [seaborn \(https://seaborn.pydata.org/\)](https://seaborn.pydata.org/). Farem servir el conjunt de dades de flors iris que ja hem vist en mòduls anteriors.

Per començar, importem les llibreries que farem servir en aquest Notebook i que ja hem presentat abans.

```
In [28]: 1 # Importem NumPy, pandas, Matplotlib i els datasets de Sklearn.  
2 import numpy as np  
3 import pandas as pd  
4 import matplotlib.pyplot as plt  
5 from sklearn import datasets  
6
```

```

In [29]: 1 # Carreguem el dataset d'iris:
          2 iris = datasets.load_iris()
          3
          4 # Guardem les dades a la variable 'data'.
          5 data = iris.data
          6
          7 # Creem un dataframe de pandas amb les dades.
          8 df = pd.DataFrame(data = np.c_[iris['data'], iris['target']],
          9                  columns = iris['feature_names'] + ['target'])
         10
         11 # Mostrem les primeres cinc files del dataframe.
         12 df.head()

```

```

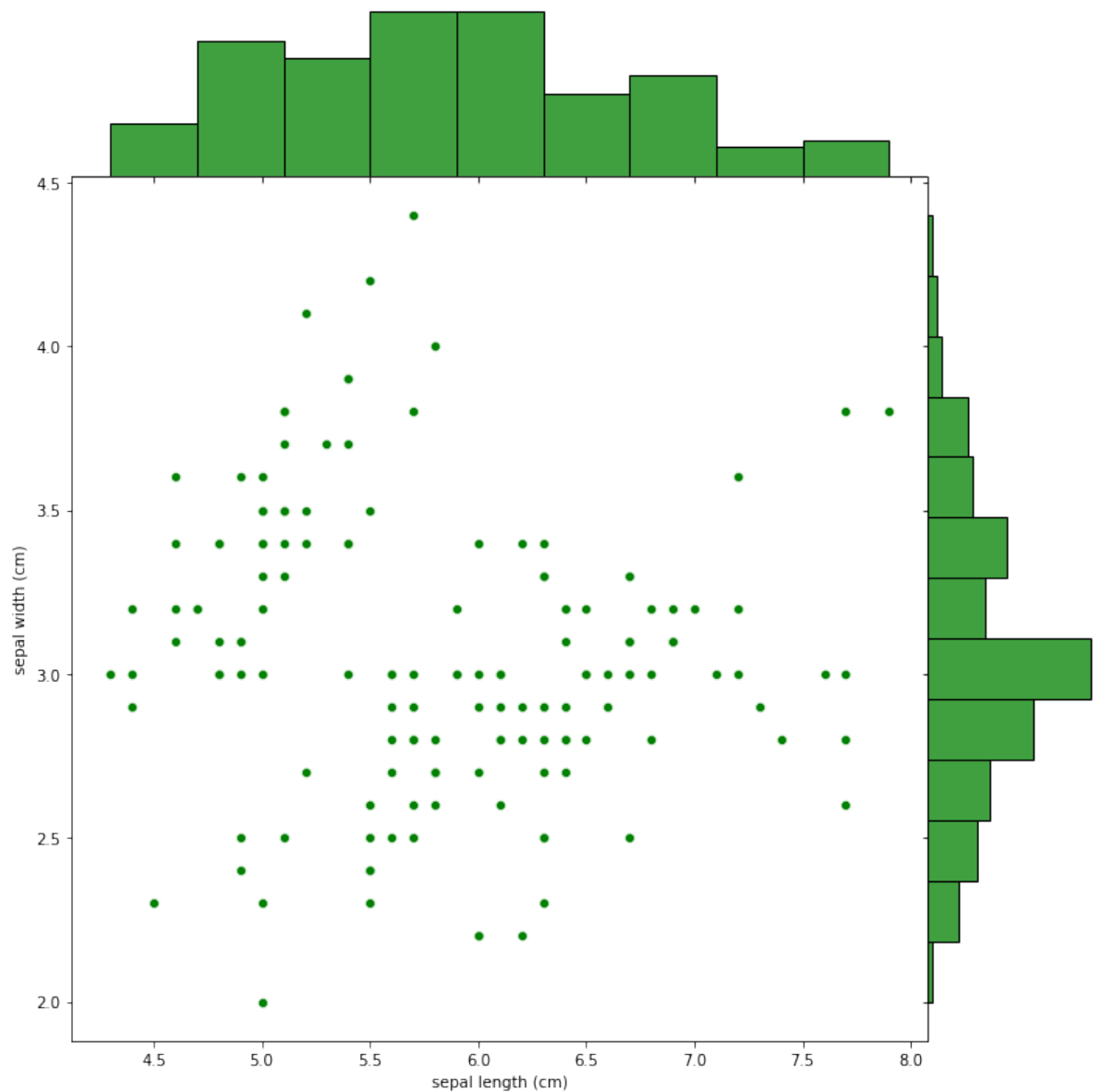
Out [29]:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1                3.5                1.4                0.2      0.0
1                4.9                3.0                1.4                0.2      0.0
2                4.7                3.2                1.3                0.2      0.0
3                4.6                3.1                1.5                0.2      0.0
4                5.0                3.6                1.4                0.2      0.0

```

La funció [jointplot](http://seaborn.pydata.org/generated/seaborn.jointplot.html) (<http://seaborn.pydata.org/generated/seaborn.jointplot.html>) permet crear una gràfica de dues variables oferint tant informació conjunta sobre les variables com marginal. La mateixa funció es pot fer servir per crear diferents tipus de gràfiques. Vegem-ne alguns exemples.

Per començar, crearem una gràfica de dispersió (en anglès, *scatter plot*) representant les mostres segons les característiques del sèpal. A més, mostrarem histogrames marginals (que representaran la distribució de totes característiques de manera individual).

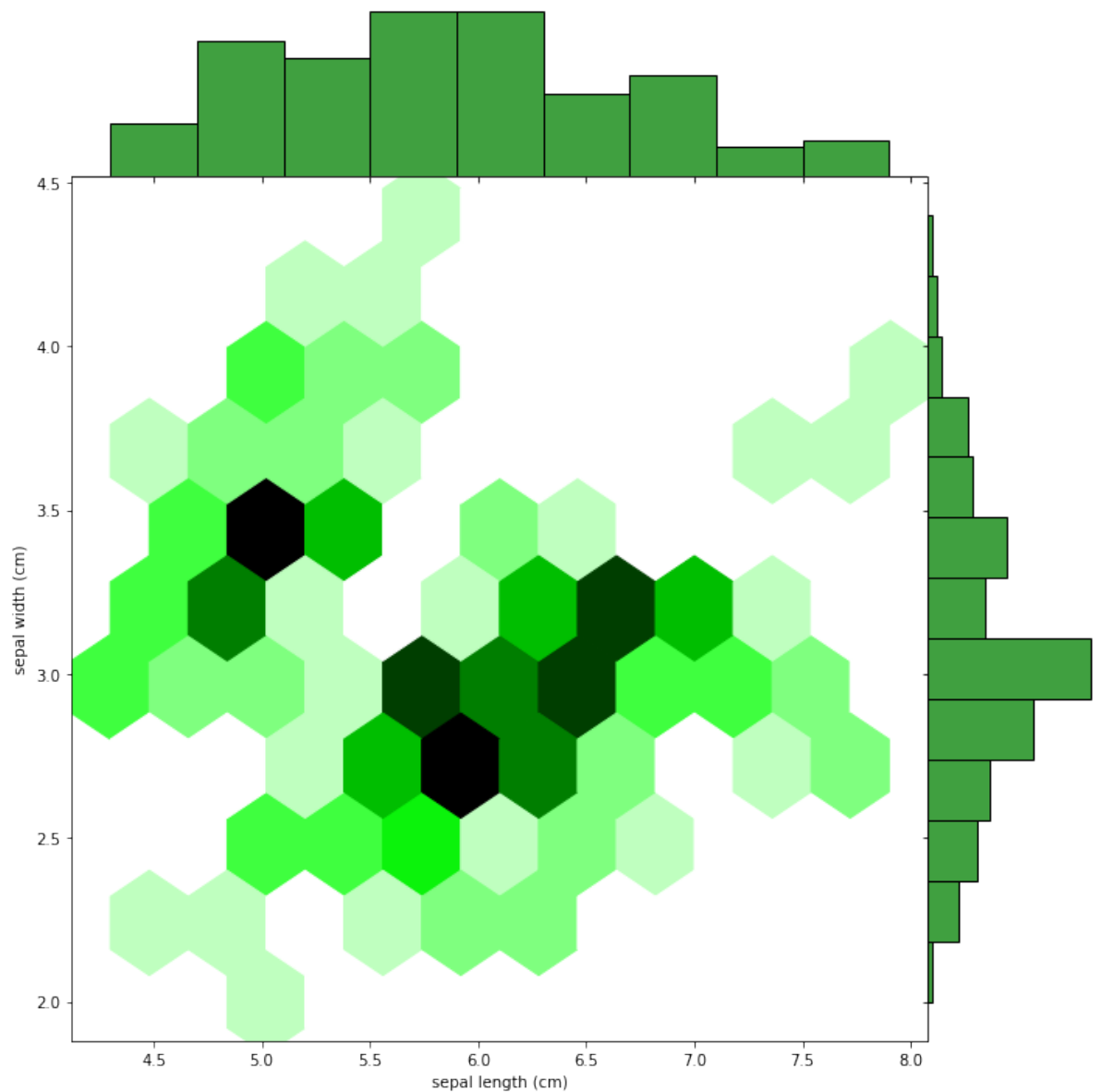
```
In [30]: 1 # Importem la llibreria seaborn.
2 import seaborn as sns
3
4 # Creem un jointplot de tipus "scatter", indicant els noms de l
5 # que volem fer servir com a variables, el color i la grandària
6 g = sns.jointplot(x = "sepal length (cm)",
7                  y = "sepal width (cm)",
8                  data=df,
9                  kind="scatter",
10                 space=0, color="g", height=10)
```



La gràfica de dispersió ens ofereix una primera aproximació a les dades: podem veure que les mostres estan distribuïdes segons la longitud i l'amplada del sèpal.

La funció `jointplot` és molt versàtil. Fent servir la mateixa funció però canviant el tipus de gràfic a `hex`, podem substituir el diagrama de dispersió per un histograma conjunt utilitzant intervals (*bins*) hexagonals.

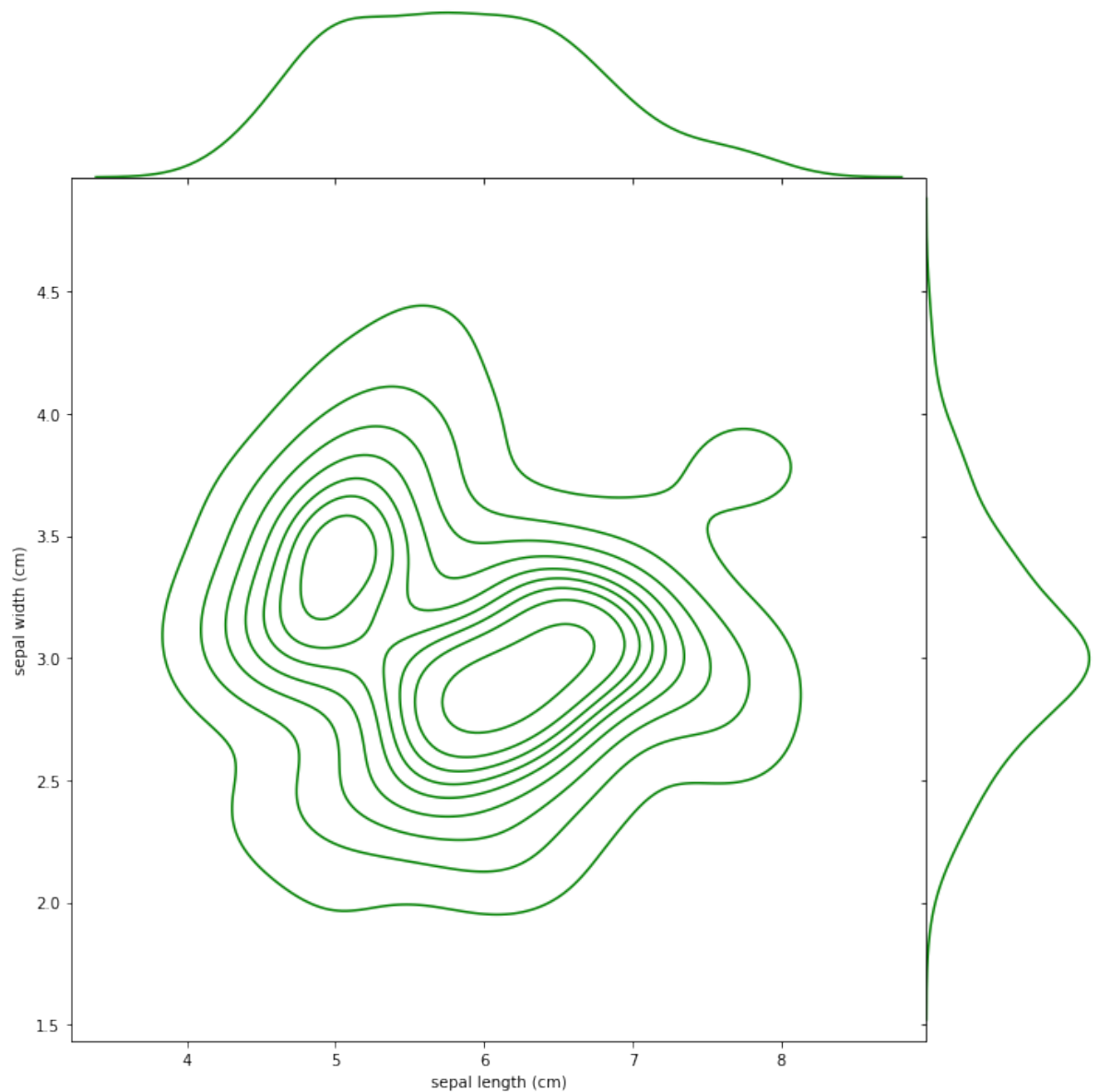
```
In [31]: 1 # Importem la llibreria seaborn.  
2 import seaborn as sns  
3  
4 # Creem un jointplot de tipus "hex", indicant els noms de les d  
5 # que volem fer servir com a variables, el color i la grandària  
6 g = sns.jointplot(x = "sepal length (cm)",  
7                  y = "sepal width (cm)",  
8                  data=df,  
9                  kind="hex",  
10                 space=0, color="g", height=10)
```



El diagrama de tipus `hex` ens permet també veure la distribució de les mostres, ometent els efectes de petites variacions i focalitzant-se, per tant, a transmetre la informació amb més granularitat.

De manera anàloga, podem crear també un diagrama amb una [estimació de la funció de densitat](https://en.wikipedia.org/wiki/kernel_density_estimation) (https://en.wikipedia.org/wiki/kernel_density_estimation) (en anglès, *KDE* o *Kernel density estimation*).

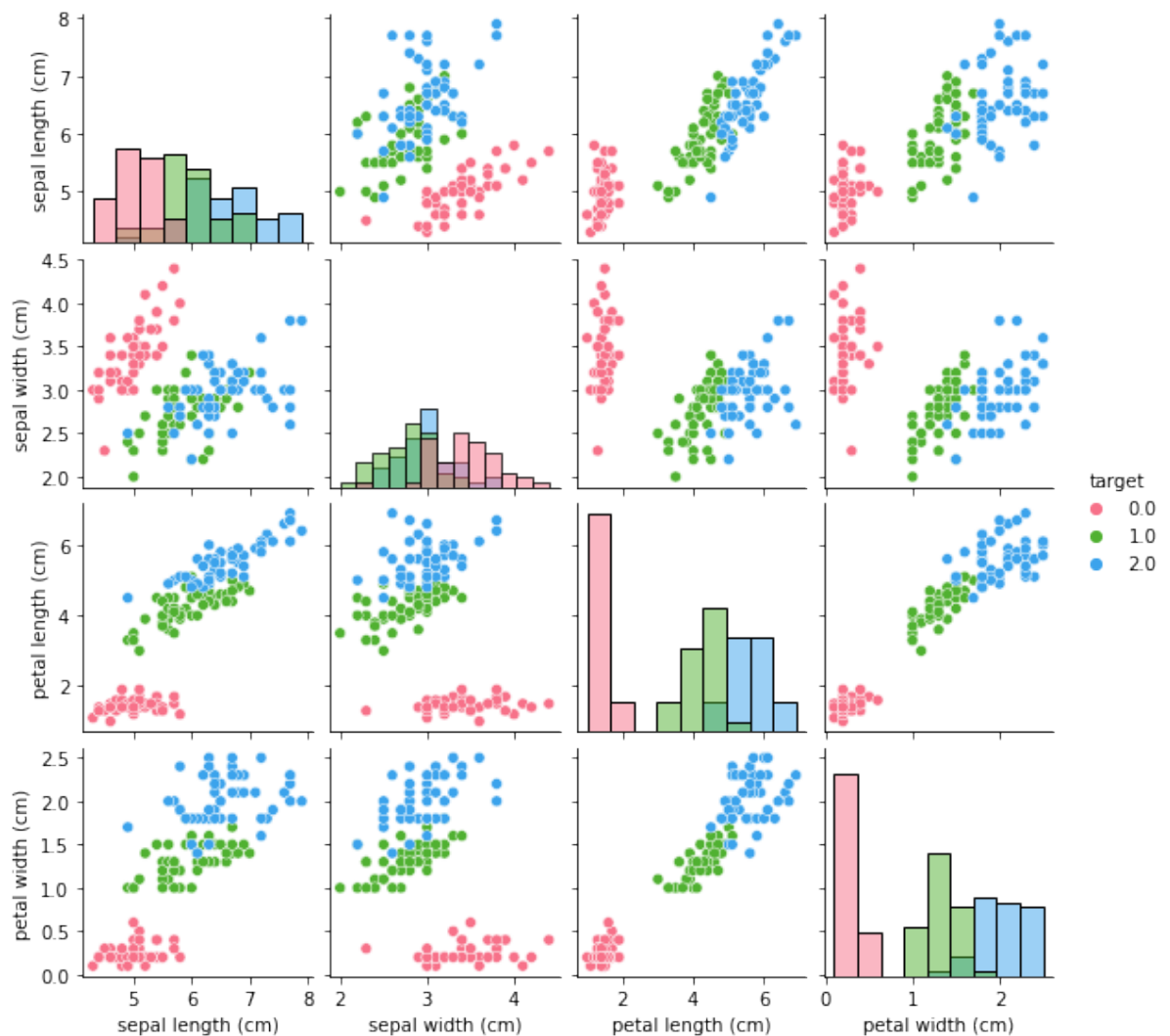
```
In [32]: 1 # Importem la llibreria seaborn.  
2 import seaborn as sns  
3  
4 # Creem un jointplot de tipus "kde", indicant els noms de les d  
5 # que volem fer servir com a variables, el color i la grandària  
6 g = sns.jointplot(x = "sepal length (cm)",  
7                  y = "sepal width (cm)",  
8                  data=df,  
9                  kind="kde",  
10                 space=0, color="g", height=10)
```



Una altra funció molt útil de la llibreria seaborn és [pairplot](http://seaborn.pydata.org/generated/seaborn.pairplot.html) (<http://seaborn.pydata.org/generated/seaborn.pairplot.html>), que crea una matriu de gràfiques amb les relacions entre parells de variables del conjunt de dades.

Així, per al conjunt d'iris que conté cinc columnes (els quatre atributs i la classe), *pairplot* generarà una matriu de 5x5 mostrant diagrames de dispersió per a cada parell de variables. A la diagonal es mostra un histograma dels valors de la variable.

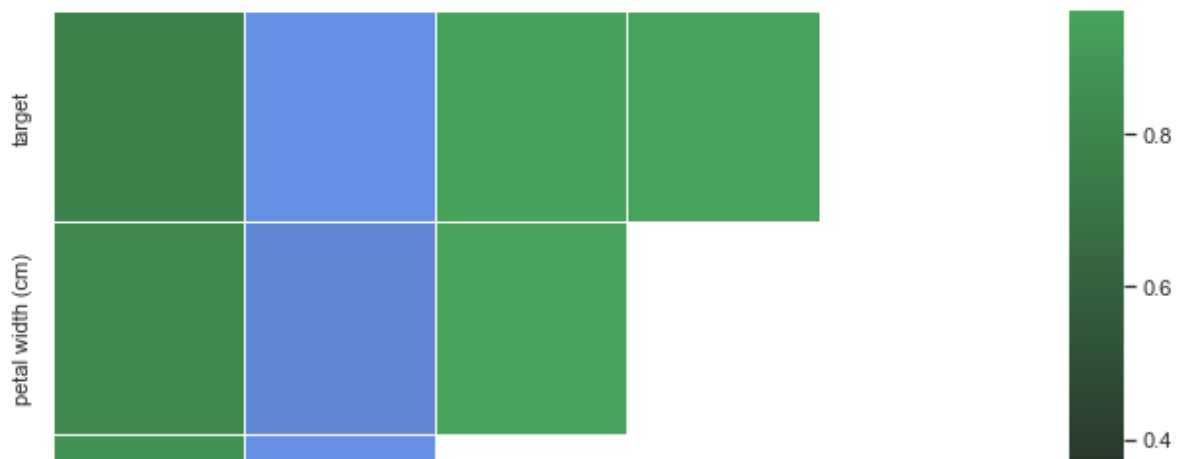
```
In [33]: 1 # Importem la llibreria seaborn.
          2 import seaborn as sns
          3
          4 # Creem un pairplot acolorint les mostres segons la classe a qu
          5 # i especificant una paleta de colors concreta.
          6 sns.pairplot(df,
          7           hue="target",
          8           height=2,
          9           palette=sns.color_palette("husl", 3),
          10          diag_kind='hist');
```

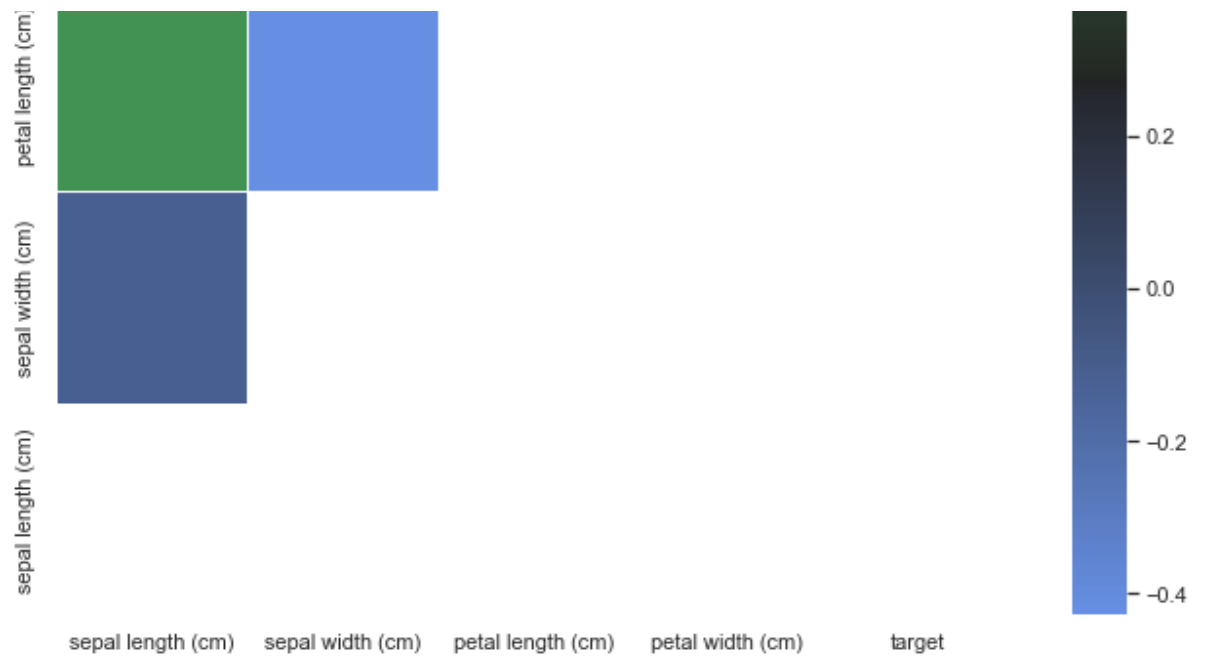


Aquesta visualització és molt útil per aproximar-nos per primera vegada a les dades. Així, per exemple, si estiguéssim afrontant un problema de classificació, podríem tenir una idea de quins atributs ens seran més útils per a la classificació o quin tipus d'algorismes ens servien per afrontar el problema.

Seaborn també ens permet crear mapes de calor o *heatmaps*. Per exemple, podem calcular la correlació entre cada parell d'atributs del nostre conjunt de dades i mostrar-ho en un mapa de calor:

```
In [34]: 1 import seaborn as sns
          2
          3
          4 sns.set(style="white")
          5
          6 # Calculem la correlació entre atributs.
          7 corr = df.corr()
          8
          9 # Generem una màscara triangular (una matriu de la mateixa gran
         10 # de correlacions, amb valors False al triangle inferior i True
         11 mask = np.zeros_like(corr, dtype=bool)
         12 mask[np.triu_indices_from(mask)] = True
         13
         14 # Generem la figura Matplotlib.
         15 f, ax = plt.subplots(figsize=(11, 10))
         16
         17 # Creem un mapa de colors.
         18 cmap = sns.diverging_palette(255, 133, l=60, n=7, center="dark")
         19
         20 # Dibuixem el mapa de calor utilitzant com a màscara la matriu i
         21 # els colors especificats.
         22 sns.heatmap(corr, mask=mask, cmap=cmap,
         23             xticklabels=list(df), yticklabels=list(df),
         24             linewidths=.5, ax=ax)
         25
         26 # Per un error a la versió de Matplotlib, cal especificar manual
         27 # els límits de l'eix y
         28 # Aquesta instrucció es podrà eliminar quan fem servir Matplotlib
         29 _ = plt.ylim((0,5))
```





Fixeu-vos que hem generat i aplicat una màscara per evitar que es mostrin valors repetits, ja que la matriu de correlació és simètrica:

```
In [35]: 1 print(mask)
```

```
[[ True  True  True  True  True]
 [False  True  True  True  True]
 [False False  True  True  True]
 [False False False  True  True]
 [False False False False  True]]
```

3 Visualització de Grafs amb Networkx

Actualment hi ha una gran quantitat de conjunts de dades que representen una xarxa, per exemple, dades d'usuaris de xarxes socials, de trucades telefòniques o enviament de missatges, interaccions entre proteïnes o col·laboració entre científics.

[Networkx \(https://networkx.github.io/\)](https://networkx.github.io/) no pot considerar-se una llibreria de visualització de dades: el seu objectiu és proporcionar estructures de dades i algorismes que treballin sobre [grafs \(https://en.wikipedia.org/wiki/graph_\(discrete_mathematics\)\)](https://en.wikipedia.org/wiki/graph_(discrete_mathematics)), permetent l'estudi de xarxes. De totes maneres, la llibreria inclou funcions bàsiques per a la visualització de grafs i és molt senzilla de fer servir, per això en aquest mòdul veurem alguns exemples de com visualitzar grafs amb Networkx, ajustant propietats (com la grandària i el color dels nodes i de les arestes) a les propietats del graf que ens interressi destacar.

Hi ha altres llibreries que permeten especificar amb molt més detall els dibuixos dels grafs que crearem, però són una mica més costoses d'aprendre. A més a més, sovint la generació de visualitzacions de grafs es fa amb programes externs especialitzats, com per exemple, [Gephi \(https://gephi.org/\)](https://gephi.org/) o [Cytoscape \(http://www.cytoscape.org/\)](http://www.cytoscape.org/).

Podem generar una representació gràfica d'un graf [Networkx \(https://networkx.github.io/\)](https://networkx.github.io/) amb la funció `draw`.

```
In [36]: 1 # Importem la llibreria Networkx.  
2 import networkx as nx  
3  
4 # Importem el graf del club de karate Zachary.  
5 G = nx.karate_club_graph()  
6  
7 # Mostrem el graf.  
8 nx.draw(G)
```

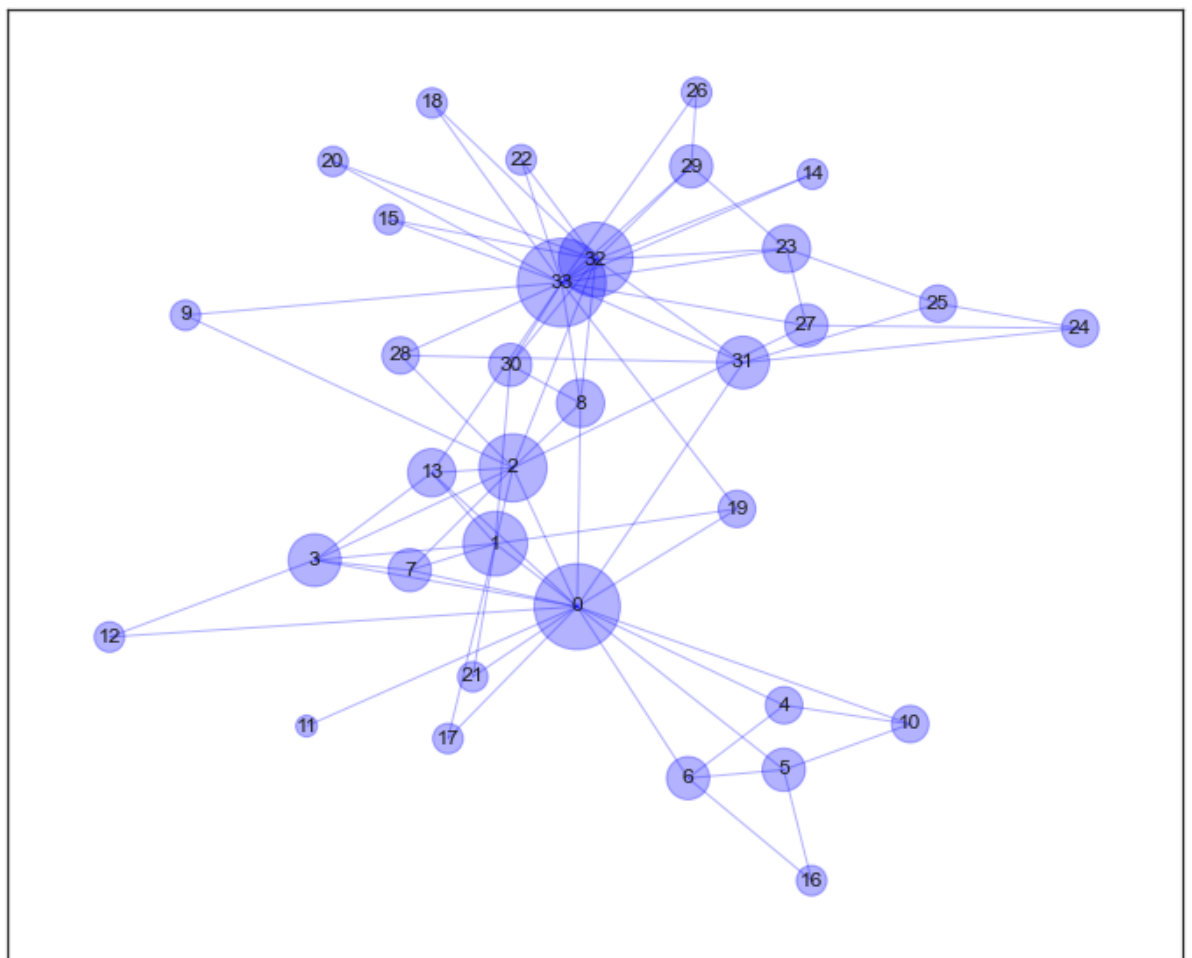


Networkx permet ajustar la visualització del graf seleccionant els colors i grandàries dels nodes i arestes i decidint la posició de cada node al plànol en funció d'un algorisme concret. Generarem una representació gràfica del graf anterior que representi el grau dels nodes amb la grandària.

```

In [37]: 1 # Generem una nova figura.
          2 plt.figure(1, figsize=(12, 10))
          3
          4 # Calculem les posicions dels nodes del graf al plànol amb l'algorisme
          5 # spring.
          6 graph_pos = nx.spring_layout(G)
          7
          8 # Calculem el grau dels nodes del graf.
          9 d = nx.degree(G)
         10
         11 # Mostrem els nodes del graf, especificant la posició, la grandària
         12 # el color i la transparència.
         13 nx.draw_networkx_nodes(G, graph_pos,
         14                        node_size=[v[1] * 150 for v in d],
         15                        node_color='blue',
         16                        alpha=0.3)
         17
         18 # Mostrem les arestes del graf especificant-ne la posició,
         19 # el color i la transparència.
         20 nx.draw_networkx_edges(G, graph_pos, edge_color='blue', alpha=0.3)
         21
         22 # Mostrem les etiquetes indicant-ne la font i la grandària.
         23 a = nx.draw_networkx_labels(G, graph_pos, font_size=12, font_fa

```



També podem fer servir el color dels nodes per representar-ne propietats. Per exemple, el color es pot fer servir per representar la comunitat a què pertanyen. La comunitat a què pertany cada node pot obtenir-se de l'execució d'un algorisme de [detecció de comunitats](https://en.wikipedia.org/wiki/community_structure) (https://en.wikipedia.org/wiki/community_structure).

```

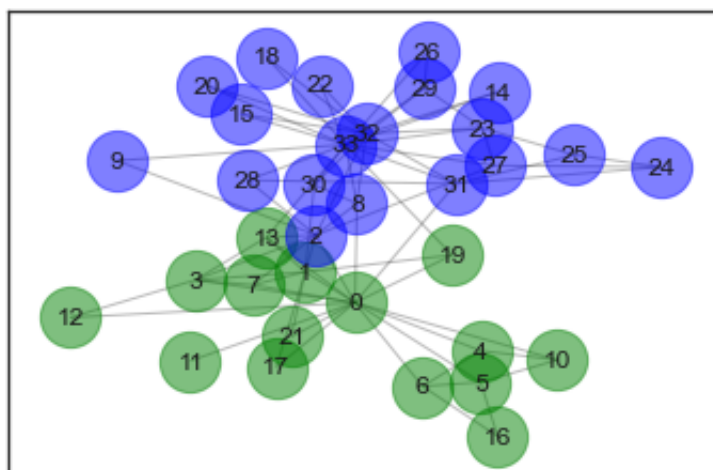
In [38]: 1 # Importem la llibreria de detecció de comunitats.
          2
          3
          4 from networkx.algorithms import community
          5 from networkx.algorithms.community centrality import girvan_newman
          6
          7 # Detectem les comunitats que hi ha al graf.
          8
          9 communities = girvan_newman(G)
         10
         11 node_groups = []
         12 for com in next(communities):
         13     node_groups.append(list(com))
         14
         15 print(node_groups)
         16
         17
         18 # Definim els colors que farem servir per als nodes.
         19 colors = ['green', 'blue', 'red', 'orange', 'yellow', 'magenta']
         20
         21 # Per a cada comunitat detectada, en mostrem els nodes:
         22 for count, com in enumerate(node_groups):
         23     # Mostrem els nodes, acolorits segons la comunitat a què pe
         24     nx.draw_networkx_nodes(G, graph_pos, com,
         25                             node_size = 800,
         26                             node_color = colors[count],
         27                             alpha = 0.5)
         28
         29 # Mostrem les arestes del graf especificant-ne la posició,
         30 # el color i la transparència.
         31 nx.draw_networkx_edges(G, graph_pos, edge_color='k', alpha=0.3)
         32
         33 # Mostrem les etiquetes indicant-ne la font i la grandària.
         34 a = nx.draw_networkx_labels(G, graph_pos, font_size=12, font_fa
         35

```

```

[[0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21], [2, 8, 9,
14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]]

```



4 Visualització de geolocalització

En aquest apartat veurem potser tres elements que ens ajudaran a treballar i visualitzar elements de geolocalització. Per això veurem primer el format de GeoJSON , geopandas i finalment folium .

4.1 GeoJSON

El format [GeoJSON \(http://geojson.org/\)](http://geojson.org/) permet codificar una varietat d'estructures de dades geogràfiques fent servir notació JSON. Amb GeoJSON es poden representar diferents objectes geomètrics com punts, línies o polígons, entre d'altres. A més, es poden assignar característiques addicionals als objectes geomètrics i representar-ne conjunts.

Els objectes GeoJSON han de tenir la clau `type` , que defineix el tipus d'estructura geomètrica que es representa. Les coordenades geogràfiques s'inclouen a la clau «coordinates» seguint la convenció d'expressar primer la longitud i després la latitud. De manera opcional, també poden incloure altitud o elevació.

Així, per exemple, un punt es representa de la manera següent:

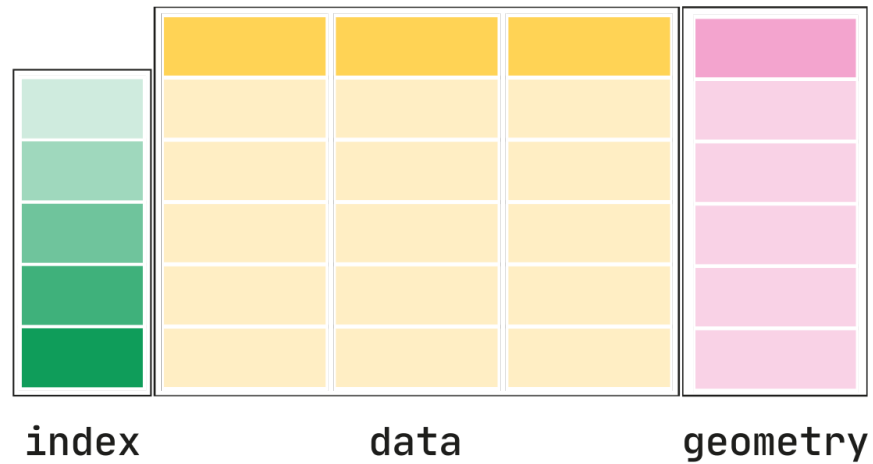
```
{
  "type": "Point",
  "coordinates": [100.0, 0.0]
}
```

I un polígon:

```
{
  "type": "Polygon",
  "coordinates": [
    [
      [100.0, 0.0],
      [101.0, 0.0],
      [101.0, 1.0],
      [100.0, 1.0],
      [100.0, 0.0]
    ]
  ]
}
```

4.2 geopandas

GeoPandas, com el seu nom indica, amplia la biblioteca de ciències de dades `pandas` que ja hem vist a l'assignatura, tot afegint suport per a dades geoespacial.



L'estructura de dades bàsica de GeoPandas és `geopandas.GeoDataFrame`, una subclasse de `pandas.DataFrame`, que pot emmagatzemar columnes de geometria i fer operacions espacials.

Al seu torn, `geopandas.GeoSeries`, una subclasse de `pandas.Series`, gestiona les geometries. El `GeoDataFrame` és, per tant, una combinació de `pandas.Series`, amb dades tradicionals (numèriques, booleanes, text, etc.) i `geopandas.GeoSeries`, amb geometries (punts, polígons, etc.). Podeu tenir tantes columnes amb geometries com vulgueu.

Veureu al codi que hi ha un mòdul anomenat `shapely` que ens proporciona classes per als diferents objectes geomètrics que necessitem fer servir, com per exemple, un punt (`shapely.geometry.Point`).

```
In [39]: 1 # Importem la llibreria geopandas.
          2 import geopandas as gpd
          3 import pandas as pd
          4
          5 from shapely.geometry import Point
          6
          7 # Carreguem el fitxer bus.csv, que conté coordenades geogràfiques
          8 df = pd.read_csv('data/bus.csv')
          9
          10 df.head()
          11
```

```
Out [39]:
```

	name	lat	lon
0	Rådhuspassagen	55.743933	12.493921
1	Fortvej (Rødovre)	55.695224	12.448593
2	Amagerhallen	55.623125	12.614444
3	Københavns Lufthavn, Kastrup (fjernbus)	55.629227	12.645824
4	Valby mod Kastrup (fjernbus)	55.663519	12.515759

Ara que tenim informació de latitud i longitud podem crear punts.

Un punt és essencialment un únic objecte que descriu la longitud i la latitud d'un punt de dades. L'ús de la comprensió de llista ens permetrà fer-ho en una línia, però assegureu-vos d'especificar sempre la columna `Longitud` abans de la columna `Latitud`:

```
In [40]: 1 geometria = [Point(xy) for xy in zip(df.lon, df.lat)]
          2 geometria[:5]
```

```
Out [40]: [<shapely.geometry.point.Point at 0x16460ebf0>,
<shapely.geometry.point.Point at 0x165597250>,
<shapely.geometry.point.Point at 0x165597100>,
<shapely.geometry.point.Point at 0x165595fc0>,
<shapely.geometry.point.Point at 0x1655955d0>]
```

Es pot veure que un punt és, de fet, un punt:

```
In [41]: 1 geometria[0]
```

```
Out [41]: <shapely.geometry.point.Point at 0x16460ebf0>
```

I ara podem crear un `GeoDataFrame`. Veureu que fem servir un paràmetre que indica com hem d'interpretar les coordenades en funció de la codificació indicada per [CRS](https://geopandas.org/en/stable/docs/user_guide/projections.html) (https://geopandas.org/en/stable/docs/user_guide/projections.html). Aquí habitualment indicarem el codi "EPSG:4326", per indicar l'estàndard *WGS84 Latitude/Longitude*.

```
In [42]: 1 gdf = gpd.GeoDataFrame(df, # El dataframe de pandas normal
2                                crs = 'EPSG:4326',
3                                geometry = geometria )
4 # La geometria en aquest cas és una llista de punts
5
6 gdf.head()
7 # Com veiem, ara hem afegit a la columna de geometria,
8 # específica de geopandas, la informació de punts
9
```

Out [42]:

	name	lat	lon	geometry
0	Rådhuspassagen	55.743933	12.493921	POINT (12.49392 55.74393)
1	Fortvej (Rødovre)	55.695224	12.448593	POINT (12.44859 55.69522)
2	Amagerhallen	55.623125	12.614444	POINT (12.61444 55.62313)
3	Københavns Lufthavn, Kastrup (fjernbus)	55.629227	12.645824	POINT (12.64582 55.62923)
4	Valby mod Kastrup (fjernbus)	55.663519	12.515759	POINT (12.51576 55.66352)

podem emprar el mètode `.explore()` per mostrar els punts de l'objecte anterior sobre un mapa.

El mètode és força configurable i en podeu trobar més detalls al [manual](https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.explore.html)
[https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.explore.ht](https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.explore.html)
 En aquest cas, modifiquem la funció per canviar el color i les propietats dels marcadors.

```
In [43]: 1 gdf.explore("name",
2           legend=False,
3           marker_kwds={"fill": "False",
4                        "radius": 0.5},
5           style_kwds={"stroke": False,
6                       "fillColor": "blue",
7                       "opacity": 0.2})
```

Out [43]: Make this Notebook Trusted to load map: File -> Trust Notebook

4.2.1 Àrees

Ara farem servir un *dataset* amb els barris de Nova York. Aquest *dataset* ve d'exemple a GeoPandas i el podem importar de la manera següent:

```
In [44]: 1 import geopandas as gpd
          2
          3 path_to_data = gpd.datasets.get_path("nybb")
          4 gdf = gpd.read_file(path_to_data)
          5
          6 gdf
```

```
Out [44]:
```

	BoroCode	BoroName	Shape_Leng	Shape_Area	geometry
0	5	Staten Island	330470.010332	1.623820e+09	MULTIPOLYGON (((970217.022 145643.332, 970227....
1	4	Queens	896344.047763	3.045213e+09	MULTIPOLYGON (((1029606.077 156073.814, 102957...
2	3	Brooklyn	741080.523166	1.937479e+09	MULTIPOLYGON (((1021176.479 151374.797, 102100...
3	1	Manhattan	359299.096471	6.364715e+08	MULTIPOLYGON (((981219.056 188655.316, 980940....
4	2	Bronx	464392.991824	1.186925e+09	MULTIPOLYGON (((1012821.806 229228.265, 101278...

Com veieu, aquest *dataset* conté cinc àrees de la zona de NY, en què a la columna geometria hi tenim, de fet, la definició de les àrees de cada zona, en comptes de punts com a l'exemple anterior. Podem veure què conté el segon element de la columna geometria (Queens), en què de fet, veurem la representació de l'objecte *multipolygon*.

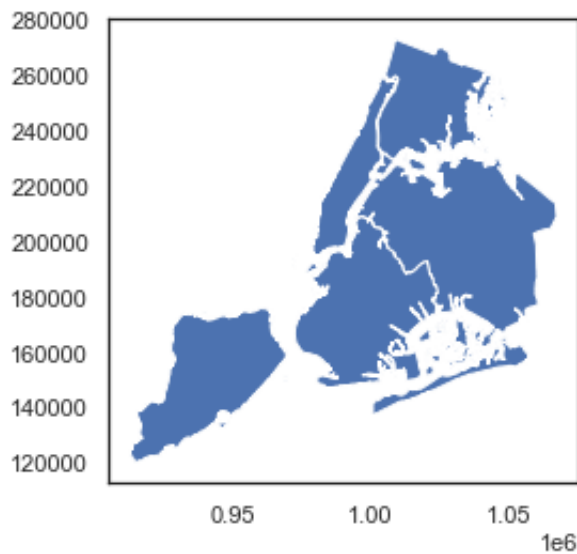
```
In [45]: 1 gdf.geometry[1]
```

```
Out [45]: <shapely.geometry.multipolygon.MultiPolygon at 0x168a675e0>
```

podem representar tots el barris com en l'exemple anterior, o bé amb plot


```
In [46]: 1 gdf.plot()
```

```
Out[46]: <AxesSubplot:>
```



o bé amb el mètode `explore` :

```
In [47]: 1 gdf.explore("BoroName", legend=True)
```

```
Out[47]: Make this Notebook Trusted to load map: File -> Trust Notebook
```

Per mesurar l'àrea de cada polígon (o MultiPolygon en aquest cas concret), podem utilitzar l'atribut `GeoDataFrame.area`, que retorna un `pandas.Series`.

Podem trobar els atributs directament de l'objecte `GeoDataFrame` :

```
In [48]: 1 gdf.area
```

```
Out[48]: 0    1.623822e+09
         1    3.045214e+09
         2    1.937478e+09
         3    6.364712e+08
         4    1.186926e+09
         dtype: float64
```

Si fixem l'índex del dataframe als noms dels barris, la sortida és més interpretable:

```
In [49]: 1 gdf = gdf.set_index("BoroName")
         2 gdf.area
```

```
Out[49]: BoroName
         Staten Island    1.623822e+09
         Queens          3.045214e+09
         Brooklyn       1.937478e+09
         Manhattan      6.364712e+08
         Bronx          1.186926e+09
         dtype: float64
```

4.2.2 Centroides i càlcul de distàncies

Donat un polígon (o un multipolígon, en aquest cas), podem calcular el centroide o punt mig del polígon (representant el punt mig del barri, o la coordenada mitjana del barri).

Aquesta és, de fet, una de les propietats que podem llegir, per exemple:

```
In [50]: 1 queens = gdf.loc["Queens", "geometry"].centroid
         2 brooklyn = gdf.loc["Brooklyn", "geometry"].centroid
```

```
In [51]: 1 type(queens)
```

```
Out[51]: shapely.geometry.point.Point
```

```
In [52]: 1 str(queens)
```

```
Out[52]: 'POINT (1034578.0784064555 197116.60422991414) '
```

```
In [53]: 1 queens
```

```
Out[53]: <shapely.geometry.point.Point at 0x168a044f0>
```

Amb dos punts en les dues variables, ja podem, de fet, calcular la distància entre els centroides dels dos barris. Això és ben fàcil de fer mitjançant el mètode `distance`.

```
In [54]: 1 queens.distance(brooklyn)
```

```
Out [54]: 42530.453903934955
```

4.3 Folium

`folium` facilita la visualització de les dades que s'han manipulat a Python en un mapa interactiu. Permet tant l'enllaç de dades a un mapa per a visualitzacions, com incloure marcadors en el mapa i altres objectes com ara imatges, vídeos, GeoJSON i TopoJSON .

Així mateix, també admet l'ús de sèrie d'OpenStreetMap, Mapbox i Stamen, i, amb claus API, l'ús de Mapbox o Cloudmade.

La manera més senzilla de representar un mapa és a partir d'unes coordenades:

```
In [55]: 1 import folium
          2
          3 B = folium.Map(location = [41.38879, 2.15899])
```

per representar el mapa senzillament avaluarem la variable,

```
In [56]: 1 B
```

```
Out [56]: Make this Notebook Trusted to load map: File -> Trust Notebook
```

Es poden incloure marcadors amb la funció `Marker` , per exemple: si volem incloure un marcador sobre la Sagrada Família de Barcelona (Lat = 41.403706, Lon= 2.173504), faríem:

```
In [57]: 1 folium.Marker(  
2         [41.403706, 2.173504],  
3         popup="<i>Sagrada Família</i>",  
4         tooltip="Clica per veure les propietats"  
5     ).add_to(B);  
6     # Podem afegir tants marcadors com vulguem
```

```
In [58]: 1 B
```

Out [58]: Make this Notebook Trusted to load map: File -> Trust Notebook

`folium` també ens deixa canviar els colors o fins i tot les icones dels marcadors, això ho podem fer amb l'argument `icon` . Cal tenir en compte que tenim realment moltes icones disponibles, les podeu trobar anant a l'ajuda d'`Icon` amb `help(folium.Icon)` .

També podem afegir-hi altres elements com, per exemple, marcadors circulars, o cercles, amb `CircleMarker` i `Circle` (el primer té propietats de marcador, el segon és senzillament un cercle).

```
In [59]: 1 # Com podem canviar les icones
2 folium.Marker(
3     location=[41.3857, 2.1639],
4     popup="Plaça Universitat",
5     icon=folium.Icon(color="red", icon="info-sign"),
6 ).add_to(B)
7
8 # incloure un cercle
9 folium.Circle(
10     radius=1000,
11     location=[41.3794, 2.1406],
12     popup="Sants Estació",
13     color="green",
14     fill=False
15 ).add_to(B)
16
17 # incloure un Marcador d'àrea circular
18 folium.Circle(
19     radius=1000,
20     location=[41.3838, 2.1180],
21     popup="Zona Universitaria",
22     color="green",
23     fill=False
24 ).add_to(B)
25
26 B
```

Out [59]: Make this Notebook Trusted to load map: File -> Trust Notebook

5 Visualització interactiva i creació de quadres de comandament o dashboards

De moment, començarem per explicar amb més detall les llibreries de visualització interactiva: `bokeh` i `ipywidgets`.

5.1 bokeh

`bokeh` (<https://bokeh.org>) és una llibreria de codi obert que facilita la creació de trames comunes, però també pot gestionar casos d'ús personalitzats o especialitzats.

Bokeh representa les seves gràfiques fent servir HTML i JavaScript que utilitzen navegadors web moderns per presentar una construcció elegant amb interactivitat d'alt nivell.

Algunes de les característiques de Bokeh són:

- Flexibilitat: el bokeh es pot fer servir per a visualitzacions senzilles i també complexes.
- Productivitat: la seva interacció amb altres eines populars de Pydata (com Pandas i el quadern Jupyter) és molt fàcil.
- Interactivitat: capacitat de visualitzacions interactives.
- Compatible: les dades visuals es poden compartir i també es poden representar en quaderns de Jupyter.
- Codi obert: `bokeh` és un projecte de codi obert.

Podem posar un primer exemple amb l'ús de la directiva `line`, en què veureu que és de fet força fàcil representar una línia. Veureu que la figura és interactiva i podem moure-la amb el ratolí arrossegant a sobre de la figura.

Els passos per generar la figura són:

- Crear una instància d'un plot, amb `figure()`, i assignant-la a una variable (`graph`) en l'exemple
- Invocar el mètode de representació seleccionat dintre de la variable `graph`
- En el cas de notebooks del Jupyter, especificar que la sortida és al notebook amb `output_notebook()`
- Invocar la representació de la figura amb el mètode `show()`

```
In [60]: 1 # importing the modules
2 from bokeh.plotting import figure, output_notebook, show
3
4 # Crear una instància d'un plot
5 graph = figure(title = "Figura amb una línia")
6
7 # definim dues llistes amb les coordenades
8 x = [1, 2, 3, 4, 5]
9 y = [1, 3, 2, 4, 3]
10
11 # invoquem al mètode line()
12 graph.line(x, y)
13
14 # especificar que la sortida és al notebook i representar
15 output_notebook()
16 show(graph)
17
```

(<https://bokeh.org>) Loading BokehJS ...

Hi ha altres primitives de visualització que podem fer servir, com `rect` , `square` o `ellipse` .

```
In [61]: 1 from bokeh.plotting import figure, output_notebook, show
2
3 fig = figure(plot_width = 300, plot_height = 300)
4 fig.rect(x = 10,
5         y = 6,
6         width = 10,
7         height = 15,
8         color = "blue",
9         alpha = 0.5)
10 fig.square(x = 15,
11           y = 8,
12           size = 80,
13           color = 'red',
14           alpha = 0.3)
15 fig.ellipse(x = 5,
16            y = 6,
17            width = 30,
18            height = 10,
19            fill_color = 'green', alpha = 0.2)
20 output_notebook()
21
22
23 show(fig)
```

(<https://bokeh.org>) Loading BokehJS ...

Podem veure un exemple amb més detall en què introduïrem diverses comandes per controlar. Aquí, per exemple, farem servir `vbar` per fer un diagrama de barres de pluja diària i una línia per a l'acumulat. Veurem també com modifiquem les propietats de la figura, incloent-hi la posició de la caixa d'eines de `bokeh`.

```
In [62]: 1
2 import numpy as np
3 from bokeh.plotting import figure, show
4 from bokeh.io import output_notebook
5
6 # definir les dades a representar, en aquest cas valors de pluja
7 rain = np.random.uniform(0,1,14)*1000
8
9 days = np.linspace(1, 10, len(rain))
10
11 cumulative_rain = np.cumsum(rain)
12
13 # Creem la figura
14 fig = figure(title='pluja diària',
15              plot_height=400, plot_width=700,
16              x_axis_label='dies',
17              y_axis_label='pluja',
18              x_minor_ticks=2,
19              y_range=(0, 6000),
20              toolbar_location="below")
21
22 # Diagrama de barres per a la pluja diària
23 fig.vbar(x=days, bottom=0, top=rain,
24          color='blue', width=0.75,
25          legend_label='Diari ')
26
27 # i l'acumulada amb una línia
28 fig.line(x=days, y=cumulative_rain,
29          color='red',
30          line_width=3,
31          legend_label='Acumulat')
32
33 # llegendes a dalt a l'esquerra
34 fig.legend.location = 'top_left'
35
36 # representar
37 output_notebook()
38
39 show(fig)
```

(<https://bokeh.org>) Loading BokehJS ...

A `bokeh` també és possible recuperar les dades des d'objectes de `pandas`. Per això cal crear una *font* a partir de l'objecte `dataframe` del `pandas` amb la funció `ColumnDataSource()`.

En l'exemple següent també hi hem inclòs un exemple de com podem modificar la caixa d'eines de manera que puguem incloure les icones a mida; per exemple, podem modificar els estris de selecció de punts de manera que quan creem la figura li indiquem quins vol veure l'usuari amb la propietat `tools`.

```
In [63]: 1 import pandas as pd
2 import seaborn as sns
3 import numpy as np
4
5
6 from bokeh.models import ColumnDataSource
7 from bokeh.plotting import figure, show
8 from bokeh.io import output_notebook
9
10 # Importem un dataset de seaborn
11 df = sns.load_dataset("iris")
12
13 # Definim el nom de la font del dataframe
14 data = ColumnDataSource(df)
```

```
In [64]: 1 select_tools = ['box_select', 'lasso_select', 'poly_select', 't
2
3 fig = figure(plot_width = 500, plot_height = 400,
4             toolbar_location='below',
5             tools=select_tools)
6
7 fig.square(x='sepal_width', # Nom de la columna al dataframe
8           y='petal_length', # Nom de la columna al dataframe
9           source = data,    # Nom del la font del dataframe
10          color='royalblue',
11          selection_color='orange',
12          nonselection_color='lightgray',
13          nonselection_alpha=0.3)
14
15
16 # representar
17 output_notebook()
18
19 show(fig)
20
```

(<https://bokeh.org>) Loading BokehJS ...

5.2 ipywidgets

`ipywidgets` és un conjunt de funcions que permeten interactivitat directament dintre d'un notebook.

Aquesta llibreria proporciona una llista de ginys força comuns a les aplicacions web i als taulers de control. Entre aquests ginys, hi trobem menús desplegable, caselles de selecció o botons d'opció, per exemple. Us permetrà codificar un sistema interactiu purament desde Python i la generació de ginys i interactivitat es crea en el fons via javascript.

De manera semblant a les altres alternatives que hem descrit de visualitzacions interactives, amb `ipywidgets` no hem d'aprendre javascript per construir-les. Senzillament podem emprar Python fent servir `ipywidgets` per fer un dashboard o una interfície interactiva.

5.2.1 interact

Potser una de les funcions més interessants d' `ipywidgets` és `interact` . Mitjançant `interact` podem passar una funció com argument a la funció, i aquesta funció es tornarà automàticament interactiva. Per exemple, si definim una funció com aquesta:

```
In [65]: 1 def printme(x):
          2     print("El valor de x és %.1f, el valor del quadrat de x és %s")
```

La podem convertir ara en interactiva amb una crida a `interact`. Veureu una barra amb què podeu interaccionar amb el ratolí modificant el valor de l'argument de la funció des del notebook.

```
In [66]: 1 from ipywidgets import interact
          2 interact(printme, x=1.);
```

```
interactive(children=(FloatSlider(value=1.0, description='x', max=3.0, min=-1.0), Output()), _dom_classes=('wi...
```

Cal tenir en compte que el tipus de widget depèn del tipus de la variable d'entrada a la funció. Per veure-ho en fem una versió genèrica i provem amb diferents tipus d'entrada. Veureu que el tipus de la variable el modifiquem alterant el valor per defecte de l'argument (el `x=1.` del cas anterior).

Veiem l'efecte d'incloure un booleà i un text.

```
In [67]: 1 def printme(x):  
2         print(x)
```

```
In [68]: 1 interact(printme, x=False);  
  
interactive(children=(Checkbox(value=False, description='x'), Output()),  
_dom_classes=('widget-interact',))
```

```
In [69]: 1 interact(printme, x="hola");  
  
interactive(children=(Text(value='hola', description='x'), Output()),  
_dom_classes=('widget-interact',))
```

Fixeu-vos com el text s'actualitza a cada modificació de l'entrada volent dir que la funció es crida després de cada caràcter entrat.

Si ens fixem en la sintaxi anterior, `interact` és una funció que fa servir una funció com a argument d'entrada. Això és, de fet, semblant al comportament dels *decorators*, i de fet ens permet fer servir `interact` com un decorador.

Podeu veure més informació sobre els decoradors [aquí](https://towardsdatascience.com/how-to-use-decorators-in-python-by-example-b398328163b) (<https://towardsdatascience.com/how-to-use-decorators-in-python-by-example-b398328163b>). Això fa que la semàntica d'ús d'`interact` se simplifiqui. Veiem l'últim exemple en format decoradors.

```
In [70]: 1 @interact(x="hola")  
2 def printme(x):  
3     print(x)  
  
interactive(children=(Text(value='hola', description='x'), Output()),  
_dom_classes=('widget-interact',))
```

És interessant pensar com això en conjunció amb altres llibreries que hem vist a l'assignatura pot fer visualitzacions ràpides i prou efectives.

```
In [71]: 1 import seaborn as sns
2 from ipywidgets import interact
3 df = sns.load_dataset("diamonds")
4 df.head()
```

```
Out [71]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

```
In [72]: 1 @interact
2 def filtrar_df(column='carat', x=4):
3     return df.loc[df[column] > x]
```

```
interactive(children=(Text(value='carat', description='column'), IntSlider(value=4, description='x', max=12, m...
```

o bé amb figures com a aquest exemple:

```
In [73]: 1 import seaborn as sns
2 df = sns.load_dataset("iris")
3 @interact
4 def plot_df(x=3.5):
5     f, ax = plt.subplots(figsize=(6.5, 6.5))
6     sns.scatterplot(x="petal_width", y="petal_length",
7                     hue="species",
8                     sizes=(1, 8), linewidth=0,
9                     data=df.loc[df['sepal_length'] > x], ax=ax)
```

```
interactive(children=(FloatSlider(value=3.5, description='x', max=10.5, min=-3.5), Output()), _dom_classes=('w...
```

5.2.2 Múltiples arguments

Podem fer servir `interact` amb funcions amb dos o més arguments, i dotar d'interactivitat uns mentre d'altres els *fixem* amb l'ajut de la comanda `fixed`.

```
In [74]: 1 from ipywidgets import interact, fixed
2 def multadd(a,b,c):
3     print(a*b+c)
4
5 interact(multadd, a=1,b=2, c=fixed(10));
```

```
interactive(children=(IntSlider(value=1, description='a', max=3, m
in=-1), IntSlider(value=2, description='b', ...
```

Si volem definir un rang de valors, o bé un conjunt de valors per seleccionar, podem fer servir una **tupla amb el rang de valors** per al primer cas, o una **llista amb els possibles valors a seleccionar** per al segon. Per exemple:

```
In [75]: 1 from ipywidgets import interact, fixed
2 def multadd(a,b,op):
3     if op == "add":
4         print(a+b)
5     if op == "mult":
6         print(a*b)
7
8 interact(multadd, a=(1,10),b=[-5,-2.5,0,2.5,5], op=["add","mult"]
```

```
interactive(children=(IntSlider(value=5, description='a', max=10,
min=1), Dropdown(description='b', options=(...
```

5.2.3 Crear instàncies de widgets

El que hem vist fins ara és com `interact` crea instàncies de ginyes o eines d'interactivitat de manera automàtica, però aquests ginyes els podem crear manualment amb el mòdul `widgets`. Els ginyes que potser es fan servir més són:

- [IntSlider](https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#IntSlider)
(<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#IntSlider>),
- [FloatSlider](https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#FloatSlider)
(<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#FloatSlider>),
- [Dropdown](https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Dropdown)
(<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Dropdown>),
- [Text](https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Text)
(<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Text>), i
- [Checkbox](https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Checkbox)
(<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html#Checkbox>)

Per a cadascun d'ells podem passar un paràmetre de descripció amb cada widget, ja que crearà una etiqueta al costat del widget. Fins i tot podem passar fórmules matemàtiques en format LaTeX com a cadena al paràmetre de descripció i també crearà la fórmula.

Crear el widgets manualment ens donarà més llibertat a com dotar d'interactivitat un programa.

Podem veure l'efecte de la creació del giny i com podem controlar-ne millor els paràmetres.

```
In [76]: 1 from ipywidgets import interact, fixed, widgets
          2
          3 widgets.IntSlider(min=-10, max=30, step=1, value=10)
          4
```

```
IntSlider(value=10, max=30, min=-10)
```

i usar-lo junt amb `interact`

```
In [77]: 1 def printme(x):
          2     print(x)
          3     interact(printme, x =widgets.IntSlider(min=-10, max=30, step=1,
interact(children=(IntSlider(value=10, description='x', max=30,
min=-10), Output()), _dom_classes=('widget-...
```

```
In [78]: 1 from ipywidgets import interact, fixed, widgets
2 def printme(x):
3     print(x)
4     interact(printme, x = widgets.Dropdown(options=['Galileo', 'Brahe', 'Hubble']), value='Galileo')
5
6
```

```
interactive(children=(Dropdown(description='x', options=('Galileo', 'Brahe', 'Hubble')), value='Galileo'), Output())
```

```
In [79]: 1 from ipywidgets import interact, fixed, widgets
2
3 def printme(x):
4     print(x)
5
6     interact(printme, x = widgets.Dropdown(options=([("binari", 2), ("decimal", 10), ("hexa", 16)]), value='binari'))
```

```
interactive(children=(Dropdown(description='x', options=([("binari", 2), ("decimal", 10), ("hexa", 16)]), value='binari')), Output())
```

Hi ha altres tipus de ginyes, com, per exemple

```
In [80]: 1 from ipywidgets import interact, fixed, widgets
2
3 def printme(x):
4     print(x)
5
6     interact(printme, x = widgets.DatePicker(value=pd.to_datetime('2018-01-01 00:00:00'), description='x'))
```

```
interactive(children=(DatePicker(value=Timestamp('2018-01-01 00:00:00'), description='x'), Output()), _dom_classes=[])
```

5.2.4 Creant interfícies d'usuari

Un cop hem vist els ginyes i l'ús d' `interact`, podem crear interfícies d'usuari amb l'ajut de la funció `interactive_output`. Aquesta funció ens permet crear petites interfícies amb més control que `interact` i també ens permet separar la declaració de ginyes i la seva visualització. Veureu, en l'exemple següent, que els ginyes que es declaren només es mostren en pantalla quan es crida la funció `display`.

```
In [81]: 1 from ipywidgets import widgets
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 m = widgets.FloatSlider(min=-1,max=1,step=0.1, description="freq")
6 c = widgets.FloatSlider(min=-1,max=1,step=0.1, description="phase")
7
8 # Aquí usem HBox, per incloure els dos ginyes un al costat de l'altre
9 ui = widgets.HBox([m, c])
10
11 def plot(m, c):
12     n = np.linspace(0,10,100)
13     x = n*np.sin(n)
14     y = n*np.cos(m*n+c)
15     plt.plot(x,y)
16     plt.show()
17
18 out = widgets.interactive_output(plot, {'m': m, 'c': c})
19
```

```
In [82]: 1
2 # la interfície només apareix quan cridem a display
3 display(out, ui)
```

Output()

HBox(children=(FloatSlider(value=0.0, description='freq', max=1.0, min=-1.0), FloatSlider(value=0.0, descripti...

5.2.5 Botons

Finalment, també podem incloure botons en la interfície que ens permetin capturar un clic sobre del ratolí al botó i lligar-lo a l'execució d'alguna funció.

Com que els clics dels botons són sense estat, es transmeten des del front-end al back-end mitjançant missatges personalitzats. Mitjançant el mètode `boto_clicat`, a continuació es mostra un botó que imprimeix un missatge quan s'ha fet clic.

Per capturar impressions (o qualsevol altra mena de sortida) i assegurar-vos que es mostra, assegureu-vos d'enviar-la a un giny de sortida, que en aquest cas és `widget.Output()`. Aquest giny rebrà la dada emesa com en l'exemple:


```
In [83]: 1
2 from ipywidgets import widgets
3 button = widgets.Button(description="Clica'm !")
4 output = widgets.Output() #widget que recull la sortida
5
6 display(button, output) # Mostrem el widget
7
8 def boto_clicat(x): # funció que volem que es cridi quan cliqueu
9     with output:
10         print("Rebut!")
11         print(x)
12
13 button.on_click(boto_clicat) # Aquí lliguem l'acció de clicar e

Button(description="Clica'm !", style=ButtonStyle())

Output()
```

6 Exercicis i preguntes teòriques

La part avaluable d'aquesta unitat consisteix en el lliurament d'un fitxer IPython Notebook amb extensió IPYNB que contindrà els diferents exercicis i les preguntes teòriques que s'han de contestar. Trobareu el fitxer (prog_datasci_8_entrega.ipynb) amb les activitats a la mateixa carpeta que aquest notebook que esteu llegint.

6.1 Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive Colab Notebooks, clicant botó dret a la PAC en qüestió i fent Download. D'aquesta manera us baixareu la carpeta de la PAC comprimida en zip. Aquest és l'arxiu que heu de pujar al campus de virtual de l'assignatura.

7 Bibliografia

- [Tutorial de Seaborn \(https://seaborn.pydata.org/tutorial.html\)](https://seaborn.pydata.org/tutorial.html)
- [Galeria d'exemples de Seaborn \(https://seaborn.pydata.org/examples/index.html\)](https://seaborn.pydata.org/examples/index.html)
- [Tutorial de Networkx \(http://networkx.readthedocs.io/en/networkx-1.11/tutorial/\)](http://networkx.readthedocs.io/en/networkx-1.11/tutorial/)
- [Exemples de visualització de grafs amb Networkx \(https://networkx.readthedocs.io/en/stable/examples/drawing/index.html\)](https://networkx.readthedocs.io/en/stable/examples/drawing/index.html)
- [Guia d'usuari de GeoPandas \(https://geopandas.org/en/stable/docs.html\)](https://geopandas.org/en/stable/docs.html)
- [Guia d'usuari de bokeh \(https://docs.bokeh.org/en/latest/docs/user_guide.html#userguide\)](https://docs.bokeh.org/en/latest/docs/user_guide.html#userguide)
- [Tutorial d'introducció a ipywidgets en vídeo \(https://www.youtube.com/watch?v=HY7cf-CYs1o\)](https://www.youtube.com/watch?v=HY7cf-CYs1o)
- [Guia d'Introducció a la visualització de dades de la universitat de Duke \(http://guides.library.duke.edu/datavis/vis_types\)](http://guides.library.duke.edu/datavis/vis_types)
- [Catàleg de visualitzacions de Severino Ribecca \(http://www.datavizcatalogue.com/\)](http://www.datavizcatalogue.com/)
- [folium demo \(https://github.com/gotoariel/folium-demo\)](https://github.com/gotoariel/folium-demo)
- [folium documentation \(https://python-visualization.github.io/folium/index.html\)](https://python-visualization.github.io/folium/index.html)
- [A Dramatic Tour through Python's Data Visualization Landscape \(https://dsaber.com/2016/10/02/a-dramatic-tour-through-pythons-data-visualization-landscape-including-ggplot-and-altair/\)](https://dsaber.com/2016/10/02/a-dramatic-tour-through-pythons-data-visualization-landscape-including-ggplot-and-altair/)
- [Python data visualizations on the Iris dataset \(https://www.kaggle.com/benhamner/python-data-visualizations\)](https://www.kaggle.com/benhamner/python-data-visualizations)
- [Matplotlib tutorials \(https://matplotlib.org/stable/tutorials/index.html\)](https://matplotlib.org/stable/tutorials/index.html)
- [pyplot tutorial \(https://matplotlib.org/stable/tutorials/introductory/pyplot.html\)](https://matplotlib.org/stable/tutorials/introductory/pyplot.html)

In [84]:

```
1 %watermark
```

Last updated: 2022-07-19T00:19:35.041242+02:00

Python implementation: CPython

Python version : 3.10.5

IPython version : 8.2.0

Compiler : Clang 13.0.0 (clang-1300.0.29.30)

OS : Darwin

Release : 21.5.0

Machine : arm64

Processor : arm

CPU cores : 10

Architecture: 64bit

In [85]: 1 %watermark --iversions

```
networkx : 2.8.4
altair    : 4.2.0
seaborn   : 0.11.2
numpy     : 1.23.0
folium    : 0.12.1.post1
sklearn   : 1.1.1
pandas    : 1.4.2
ipywidgets: 7.6.5
matplotlib: 3.5.2
plotnine  : 0.8.0
geopandas : 0.11.0
```

Autors

- Autora original **Cristina Pérez Solà**, 2017.
- Actualitzat per **Cristina Pérez Solà**, 2019.
- Actualitzat per **Alexandre Perera i Lluna**, 2022.

